

Final Report

FUSIONVERGE: MULTI-MODAL TRADING DECISION SUPPORT
SYSTEM

Table of Contents

1. Introduction	4
2. Literature Review	5
2.1. The Problem: Fragmentation, Subjectivity, Cognitive Load	5
2.2. Theoretical Basis for Multi-Modal Fusion	6
2.3. Existing Multimodal System in Finance	6
2.4. Techniques Underpinning Each Model Role	8
2.4.1. Text and Sentiment Processing	8
2.4.2. Visual Chart Analysis	8
2.4.3. Time-Series Price Processing	8
2.4.4. LLM-Based Reasoning and Fusion	9
2.5. Feedback, Adaption and System Refinement	9
2.6. Synthesis and Identified Gaps	9
3. Project Design	11
3.1. Project Overview	11
3.2. Domain and Users	11
3.3. Design Justification	11
3.4. System Architecture	12
3.4.1. Stage 1: Data Ingestion	12
3.4.2. Stage 2: Per-Modality Processing	12
3.4.3. Stage 3: Fusion	13
3.4.4. Stage 4: Output and Feedback	13
3.5. Technologies and Methods	13
3.5.1. Language and Orchestration	13
3.5.2. Dashboard	13
3.5.3. Sentiment model candidates	13
3.5.4. Chart/Image model candidates	13
3.5.5. Time-series model candidates	14
3.5.6. Fusion/reasoning candidates	14
3.5.7. Data sources	14
3.6. Work Plan	14
3.7. Evaluation plan	15
3.7.1. Level 1: Per-modality model selection	15
3.7.2. Level 2: System-level evaluation	16
3.7.3. Level 3: Feedback loop evaluation	16
4. Implementation	17
4.1. Architecture and Orchestration	17
4.2. Modality 1: News/Sentiment	18
4.3. Modality 2: Chart Image Pattern Detection	19

4.4. Modality 3: Price/Time-Series	20
4.5. Modality 4: Market Positioning	22
4.6. Fusion Layer	23
4.7. Feedback Mechanism and Persistence	25
4.8. Dashboard UI and Execution Flow	27
5. Evaluation	30
5.1. Evaluation Strategy	30
5.2. Unit and Integration Testing	30
5.3. System-Level and Live Integration Testing	31
5.4. Hypothesis Testing (H1-H3)	32
5.5. Testing on Data: GRU Configuration Sweep	35
5.6. User Acceptance Testing	36
5.7. Critical Discussion	37
6. Conclusion	39
7. Appendices	41
Appendix A. Prototype Model Comparison	41
A.1. Sentiment Model comparison	41
A.2. Chart/Image Model comparison	41
A.3. Time-series Model comparison	41
A.4. Fusion Model comparison	42
Appendix B. Unit Test Results	42
B.1. sentiment.py	42
B.2. chart.py	43
B.3. timeseries.py	43
B.4. positioning.py	44
B.5. feedback.py	44
B.6. fusion.py	45
B.7. orchestrator.py	46
B.8. database.py	46
Appendix C. Representative Branch-Coverage Tests	47
Appendix D. Baseline 3 Reasoning Logs	48
Appendix E. Reasoning Coherence Scoring Rubric	48
Appendix F. Full Ten-Run Consistency Results	48
Appendix G. User acceptance Testing Materials	49
G.1. User Acceptance Questionnaire	49
G.2. Likert-Scale Results	50
G.3. Open-Ended Responses	51
8. References	52

1. INTRODUCTION

Retail traders today have access to more market data than ever before. Price charts, breaking news, sentiment signals, market positioning data and a growing range of analytical tools are all readily available. Yet access to more data has not made trading decisions more consistent or reliable. The core problem is fragmentation: these sources come from different platforms, often point in different directions and there is no unified mechanism for interpreting them together. A technical indicator may suggest a bullish trend while news sentiment points the other way. A trader who checks charts first may reach a different conclusion than one who starts with news. The problem is not a lack of data, but the absence of structured interpretation when multiple conflicting signals are present.

This fragmentation imposes a real cognitive burden. Traders must mentally aggregate inputs from different tools, weight them against each other without any formal process and arrive at a decision that is difficult to reproduce or justify consistently. The result is that decision quality varies not just with market conditions, but with the order and selection of sources consulted. This leads to outcome that has less to do with market insight than with how information happens to be presented.

FusionVerge is a multi-modal decision support system designed to address this gap, developed under project **template 4.1: Orchestrating AI Models to Achieve a Goal**. It processes four distinct data types: price time-series, technical chart images, financial news and market positioning data, through dedicated specialist models and fuses their outputs into a single structured report. The report surfaces not just an overall market stance but a per-modality signal breakdown, a confidence score and an explicit conflict level reflecting the degree of disagreement across inputs. The goal is not to automate trading decisions but to give retail traders a consistent, interpretable basis for making them.

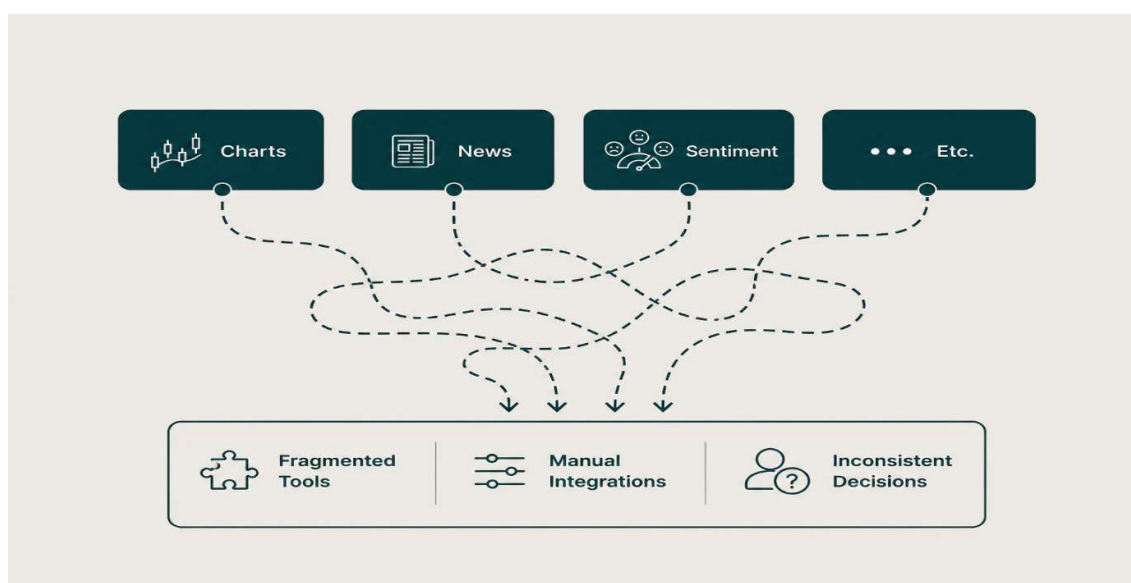


Figure 1: Retail traders access multiple data sources through fragmented tools, leading to manual integration and inconsistent decisions

2. LITERATURE REVIEW

Retail traders routinely work across multiple data streams: price charts, news, sentiment signals, market positioning data and more. Yet there are no widely accessible tool brings these together into a single, consistent output. Each stream is insightful on its own. The problem is what happens when you try to use them together. Traders end up switching platforms, weighing signals against each other with no formal process and reaching conclusions that can shift depending on which source they looked at last.

This review covers seven interconnected areas of literature relevant to building a system that addresses this gap: the cognitive and analytical limitations that make the problem real, the theoretical basis for multi-modal fusion, the landscape of existing multimodal systems in finance, the individual techniques underpinning each processing role, the use of LLMs for structured reasoning, decision support system design and adaptive feedback mechanisms. The goal is not to summarise these areas in isolation. Each body of work either reveals a limitation the proposed system must address or provides a method it draws on and the thread connecting them is traced throughout.

2.1. The Problem: Fragmentation, Subjectivity, Cognitive Load

Understanding why the problem exists requires looking at the human, signal and sentiment side.

On the human side, Roetzel and Fehrenbacher (2019) studied information overload within decision support systems and found a consistent pattern: as the number of heterogeneous inputs increases without structured support, decision quality deteriorates. What makes this finding particularly relevant is that they also found that adding more information, even high-quality information, can worsen outcomes if it is not organised in a way the user can act on. That is the trader's problem. Not a lack of data, but no structure holding it together. A system that aggregates more feeds without resolving conflicts would likely increase overload rather than reduce it.

On the signal side, the picture is just as messy. Caporale and Plastun (2023) reviewed technical indicators across a range of empirical studies and found them to be fundamentally subjective. The same chart, same indicators, different analysts, different conclusions. With the sheer number of indicators available, some will appear predictive purely by chance, what the authors call data mining bias. RSI, momentum and similar tools are backward-looking by construction and do not reliably anticipate future price movement. Strategies built on them show mixed and unstable results over time, even when individual signals appear correct. Technical analysis is not useless, but it becomes unreliable as a sole input and the problem compounds when multiple indicators conflict with no mechanism to resolve them.

Mino and Williamson (2025) add the sentiment dimension. News sentiment carries a statistically significant effect on market behaviour, particularly during uncertain periods where investor reactions follow media narratives more than underlying fundamentals. Negative sentiment correlates with increased volatility. Markets respond to the narrative layer. But that same sensitivity is what makes sentiment unreliable as a standalone signal: it is highly context-dependent and its relationship to price movement shifts with conditions, timing and data source. The paper is also

explicit that even advanced models like BERT produce outputs that reflect investor perception rather than objective fact, so sentiment functions as an indirect signal requiring interpretation in context. Individual signals are unstable and situation-specific. Without a system that integrates them deliberately, the interpretive burden falls entirely on the user.

2.2. Theoretical Basis for Multi-Modal Fusion

Before examining what existing systems have built, it is worth establishing why multi-modal fusion should work at all, not just in practice but in principle.

Huang et al. (2021) provide the first formal theoretical treatment of this question. Under a standard fusion framework where features from different modalities are encoded into a shared latent space before being mapped to an output, they prove that “learning with multiple modalities achieves a smaller population risk than only using its subset of modalities.” The intuition is straightforward: each additional modality gives a more accurate estimate of the underlying latent representation, cutting generalisation error. The authors conclude that “multi-modal learning does possess an appealing formal guarantee” relative to uni-modal approaches.

For the proposed system this matters on two levels. First, it provides a principled basis for combining price, sentiment, chart and positioning signals rather than simply arguing that more inputs are better. Second, the benefit is not purely additive. Modalities that each carry partial information about the same underlying market state together produce a more accurate picture than any single modality alone, even when each individual signal is noisy. That is not an assumption. It is what the proof shows.

2.3. Existing Multimodal System in Finance

The closest prior work comes from two recent research prototypes that both process heterogeneous financial data through specialised model roles, but with a fundamentally different end goal from the system proposed here.

FinAgent (Zhang et al., 2024) frames trading as a Markov Decision Process, explicitly targeting maximum cumulative discounted returns. Five modules run in sequence: a market intelligence module covering news, prices and financial reports; a memory module for vector-based retrieval of historical data; a low-level reflection module correlating market signals with past price movements; a high-level reflection module reviewing past trading decisions to extract lessons; and a decision-making module integrating all of this alongside MACD, KDJ and RSI to output BUY, SELL, or HOLD. The reasoning is chain-of-thought driven and passes through multiple stages. But it stays internal to the agent loop. As the paper describes, a multimodal LLM agent determines its action through integrated module outputs, with each module feeding structured prompts into the next. The output is an action. The system is built for returns, not for helping a user understand why signals agree or conflict. There is no structured explanation surface for a user to read.

MountainLion (Wu et al., 2025) takes a layered multi-agent approach targeting cryptocurrency markets. The system describes its purpose as integrating “diverse data sources, real-time market data, news sentiment and on-chain activity, into a unified AI-driven decision-making pipeline.” Specialised agents handle technical analysis, market dynamics, trading recommendations and

semantic refinement, with outputs consolidated into an investment report. Its forecasting module runs two parallel tracks: an LLM-based path using sentiment embeddings alongside structured OHLCV data and a classical ML path using ridge regression and decision trees on engineered technical features, fused through a weighted function updated by rolling accuracy. A news-driven engine adds graph-based semantic reasoning over financial news and a lightweight feedback policy refines retrieval based on user engagement signals including click-through rates and feedback ratings. MountainLion is more interpretable than FinAgent since it produces readable reports rather than bare actions and its feedback loop is closer in spirit to the weighting adjustment in the proposed system. But it is built for crypto, its output is a narrative report rather than a structured per-modality signal breakdown and conflicts between modalities are resolved internally without being surfaced to the user.

Lee and Yoo (2019) reviewed multimodal finance approaches more broadly and found that models combining multiple data sources consistently outperformed single-modality baselines, with early and intermediate fusion outperforming late fusion. They also flagged that existing implementations remain limited in data diversity and called explicitly for systems with more varied inputs and explainable outputs. Holopainen and Sarlin (2016) add the model uncertainty angle: individual models disagree by nature and structured aggregation rather than simple combination is what makes using multiple outputs robust. Both FinAgent and MountainLion handle disagreement implicitly inside their pipelines. The proposed system makes it explicit through a dedicated conflict level output that reflects the degree of disagreement across modalities at the point of decision.

System / Works	Modalities	Structured Output	Conflict Handling	User Explainability	Retail Focus	Feedback / Adaptation
TradingView	Technical Charts	X	X	X	✓	X
ChatGPT	Text + Image	X	X	✓	✓	X
AlphaSense	Text / Sentiment	Partial	X	Partial	X	X
FinAgent (Zhang et al., 2024))	Price + Text + Chart	X	Implicit	X	X	✓
MountainLion (Wu et al., 2025)	Price + Text + Chart	Partial	Implicit	Partial	X	✓
Proposed System	Price + Chart + Text + Positioning	✓	✓	✓	✓	✓

Table 1: Comparison of existing systems and prior works against proposed system across different design dimensions

✓ = addressed, X = not addressed, Partial = addressed but limited, Implicit = resolved internally

2.4. Techniques Underpinning Each Model Role

The proposed system assigns a dedicated model to each data type rather than routing everything through one. This approach is used for practical reasons, not just design preference. Hangloo and Arora (2025) survey fusion strategies across text, image and audio modalities and consistently find that intermediate fusion, integrating modalities during processing rather than at the output stage, outperforms late fusion approaches. Lee and Yoo (2019) found the same empirically in finance: fusion quality, not just the number of inputs, is what determines whether a multimodal system beats its single-modality counterparts. Concatenating outputs at the final step is not sufficient.

2.4.1. Text and Sentiment Processing

Domain-specific language models substantially outperform general-purpose ones for financial sentiment. Araci (2019) introduced FinBERT, a BERT-based model pre-trained and fine-tuned on financial corpora and showed “improvement in every measured metric on current state-of-the-art results for two financial sentiment analysis datasets.” The core finding is that financial language is specialised enough that a general model will systematically misread it. Terms like “correction”, “volatile”, or “bearish momentum” carry meanings in financial contexts that differ considerably from everyday usage. A generic LLM used for the sentiment role introduces systematic error at the first pipeline stage, which then carries through to fusion. Domain adaptation is not optional. It is a baseline requirement for reliable output. FinAgent's market intelligence module uses GPT-4 for sentiment classification without domain fine-tuning, which is a weakness the proposed system avoids by design.

2.4.2. Visual Chart Analysis

CNN-based image models extract a genuinely different signal from chart data than raw time-series processing does, even when both originate from the same underlying prices. Lin et al. (2026) compared deep learning models across line charts, candlestick charts and mixed-type charts incorporating MACD and RSI and found that mixed image inputs outperformed LSTM baselines on equivalent numerical data for short and medium-term predictions. Running both an image model on charts and a separate time-series model on price data in parallel is not redundant. They capture different things from the same source. A visual model picks up structural patterns like support zones or formation shapes that a numerical model does not represent directly and the reverse holds equally. MountainLion separates its technical analysis agent from its LLM forecasting path but does not treat chart images as a distinct visual modality.

2.4.3. Time-Series Price Processing

Fjellstrom (2022) evaluated an ensemble of LSTM networks on financial time-series classification and found the ensemble outperformed both randomly selected and index-tracking portfolios on the Stockholm OMX30 in average daily returns and volatility. LSTMs can capture temporal dependencies across sequences, showing how prices evolved over time rather than just its current state. The ensemble also reduced prediction variability, which connects to Holopainen and Sarlin's (2016) argument that aggregation across models improves robustness. MountainLion's ML forecasting track uses ridge regression and decision trees, lighter and lower-latency for a real-time crypto platform, but less capable of capturing non-linear temporal patterns. That trade-off works for their context. For a system targeting interpretable decision support across multiple asset types, LSTM-based processing is more appropriate.

2.4.4. LLM-Based Reasoning and Fusion

Kim et al. (2024) evaluated LLMs on financial statement analysis and found they handle complex multi-step financial reasoning well, in some cases outperforming human analysts when human judgements are split. That scenario is exactly what the fusion layer is built for. When modality signals disagree, a coherent interpretation is needed and the LLM layer provides it. But the paper also found that performance degrades when context is incomplete. The implication is direct: the fusion layer needs clean, well-structured outputs from each specialist model to function properly. It is not a general reasoner filling gaps. It is a structured integration component. The paper also acknowledges that whether AI improves real-world decision-making is still uncertain, which is why the system is positioned as decision support rather than an autonomous tool.

2.5. Feedback, Adaptation and System Refinement

The proposed system includes a feedback mechanism that adjusts modality signal weightings over time based on user interaction and tracked outcomes. Fazlollahi et al. (1997) define adaptive decision support systems as a class of DSS where an intelligent adaptation component “monitors the process to match support to the needs” of the user and the problem context. The architectural point is that adaptation works alongside existing data and model components rather than replacing them, adjusting how outputs are weighted and presented without retraining underlying models.

What the feedback loop does not do is equally important to state. It is not reinforcement learning. No policy is being optimised over a reward signal. FinAgent is built entirely on an MDP framework targeting maximum discounted returns, which requires a training environment and is not suited to a user-facing decision support context without substantial rework. MountainLion's feedback policy is closer: it refines retrieval and summarisation based on engagement signals including click-through rates and feedback ratings. But it relies on implicit behavioural data rather than explicit user judgements about decision quality. The proposed system takes direct user input, agreement, disagreement and outcome tracking, as the adjustment signal. That is more consistent with Fazlollahi et al.'s (1997) emphasis on the decision maker staying in control of the process, with the system adjusting in the background rather than driving the decision itself.

2.6. Synthesis and Identified Gaps

The literature points in one direction. Individual signals are unstable and context-dependent (Caporale & Plastun, 2023; Mino & Williamson, 2025). Humans cannot effectively integrate heterogeneous inputs without structured support and adding more data without structure makes this worse (Roetzel & Fehrenbacher, 2019). Individual models disagree by nature and structured aggregation is what makes combining them reliable (Holopainen & Sarlin, 2016). Multi-modal fusion is theoretically guaranteed to outperform any subset of modalities under standard conditions (Huang et al., 2021) and this holds in finance-specific empirical work (Lee & Yoo, 2019; Lin et al., 2026). The individual techniques are all validated: domain-adapted sentiment models (Araci, 2019), CNN-based visual chart analysis (Lin et al., 2026), LSTM-based time-series processing (Fjellstrom, 2022), LLM-based structured reasoning (Kim et al., 2024), adaptive feedback mechanisms (Fazlollahi et al., 1997).

What does not exist is a system that puts these together for the retail user rather than for autonomous execution. FinAgent and MountainLion are the strongest examples of what has actually been built. Both are technically solid. But FinAgent's architecture terminates in a trading action. MountainLion produces readable reports but resolves signal conflicts internally and does not expose them. Lee and Yoo (2019) called explicitly for more interpretable systems with broader data coverage. That has not been answered by any accessible user-facing tool. Table 1 makes this clear: no reviewed system combines explicit conflict handling, structured per-modality output, retail user focus and adaptive feedback within a single pipeline.

The proposed system treats signal conflict as a first-class output rather than an internal detail. Disagreement is surfaced through a conflict level alongside per-modality signals and a reasoning summary. The user gets not just a conclusion but enough information to evaluate it. The architecture targets the trading domain, but the approach generalises to any context where decision-makers need to integrate competing signals from heterogeneous sources under time pressure.

3. Project Design

3.1. Project Overview

This project is based on template *4.1: Orchestrating AI Models to Achieve a Goal*. The system is a multi-modal decision support tool for retail forex and equity traders. It ingests four distinct data types: price time-series, technical charts, financial news and market positioning data; processes each through a dedicated pre-trained model and fuses the outputs into a single structured decision report. The final output is not a chatbot response or a trade execution signal. It is a consistent, structured format covering market condition, overall stance, confidence score, conflict level, per-modality signals and a reasoning summary.

The system is designed as a controlled pipeline rather than a free-form agent. Data flows in one direction: ingest, process per modality, fuse, output. A lightweight feedback mechanism allows signal weightings to adjust over time based on user reactions and tracked outcomes, borrowing the structural logic of adaptive systems without implementing full reinforcement learning.

3.2. Domain and Users

The domain is financial trading, specifically decision support for retail traders operating in forex and equity markets. Retail traders routinely monitor multiple heterogeneous data streams simultaneously: price movements, technical chart patterns, breaking news and crowd positioning data. No widely accessible tool integrates these into a single, consistent output. Traders currently switch between platforms, combine signals mentally without a formal mechanism and reach conclusions that can vary depending on which source they consulted last.

The target user is a retail trader with a basic to intermediate understanding of markets. This user understands what RSI or a candlestick pattern means but struggles to form a consistent view when signals from different sources conflict. They are not quantitative analysts or algorithmic traders. They want interpretable output they can act on, not a black box prediction.

This user definition directly shapes the system's design priorities: structured and readable output over raw model scores, explicit conflict signalling rather than silent resolution and a dashboard interface that requires minimal technical knowledge to operate.

3.3. Design Justification

The decision to process each modality through a dedicated model rather than a single general-purpose model is grounded in both theoretical and empirical evidence. Huang et al. (2021) prove that multi-modal fusion achieves strictly lower population risk than any subset of modalities alone. Lee and Yoo (2019) confirm this empirically in finance, finding that intermediate fusion consistently outperforms single-modality approaches. Each model role in the proposed system extracts features that others cannot: a sentiment model reads narrative signals in news text, a chart model identifies visual pattern structure and a time-series model captures temporal price dynamics. These are genuinely complementary, not redundant.

The explicit conflict level output is a direct design response to the user problem. Existing systems such as FinAgent (Zhang et al., 2024) and MountainLion (Wu et al., 2025) resolve signal disagreement internally within their agent loops. Neither surfaces it to the user. For a retail trader who needs to understand the basis of a decision before acting on it, this is a critical gap. Making conflict a first-class output rather than an internal implementation detail is the core design differentiator of this system.

Retail traders make repeated decisions over time, returning to the same system across multiple sessions and market conditions. A system that does not adapt to observed outcomes provides no improvement in the consistency of support it delivers over time, which is precisely the gap feedback addresses. The feedback mechanism is grounded in the adaptive decision support systems framework described by Fazlollahi et al. (1997), where an adaptation component monitors the decision process and adjusts support without replacing the underlying models. This is not reinforcement learning. There is no policy being optimised over a reward signal. The system adjusts modality weightings based on explicit user feedback and tracked outcomes, keeping the user in control of the process.

The choice of a dashboard interface with a single trigger action, select asset and click analyse, is justified by the user definition. Retail traders are not developers. The interface must look and behave like a real tool, not a research prototype or a chatbot.

3.4. System Architecture

The system operates as a four-stage pipeline:

3.4.1. Stage 1: Data Ingestion

- Price and OHLCV data: pulled via yfinance for the selected asset and timeframe
- Chart image: generated programmatically from OHLCV data using mplfinance, producing a standardised candlestick chart passed to the image model
- News: fetched via RSS feeds from financial news sources including FXStreet and ForexLive, filtered by asset keyword
- Positioning data: retrieved from the MyFxBook public community outlook API, providing aggregated long/short ratios across retail brokers

3.4.2. Stage 2: Per-Modality Processing

Each data type is processed by a dedicated model:

- News/text: FinBERT-based sentiment model producing a sentiment score and directional signal
- Chart/image: candlestick pattern recognition model producing pattern classifications and confidence scores
- Price/time-series: Long Short-Term Memory (LSTM)/Gated Recurrent Unit (GRU)-based model producing a directional probability and price trend signal
- Positioning: rule-based processing of long/short ratio data producing a crowd positioning signal, noting that retail positioning is often contrarian

3.4.3. Stage 3: Fusion

A locally hosted LLM receives structured outputs from all four specialist models as a formatted prompt. It produces the final structured decision output: market condition, overall stance, confidence score, conflict level, per-modality signal summary and reasoning text. The conflict level is computed from the degree of disagreement across modality signals before being passed to the LLM for contextualisation and reasoning. Which produces an overall directional stance and language summary justifying the output of the given signal mix.

3.4.4. Stage 4: Output and Feedback

Results are displayed on a web dashboard. The user can provide explicit feedback (agree/disagree, outcome tracking). Feedback is logged and used to adjust modality weightings in subsequent analyses through a simple scoring mechanism.

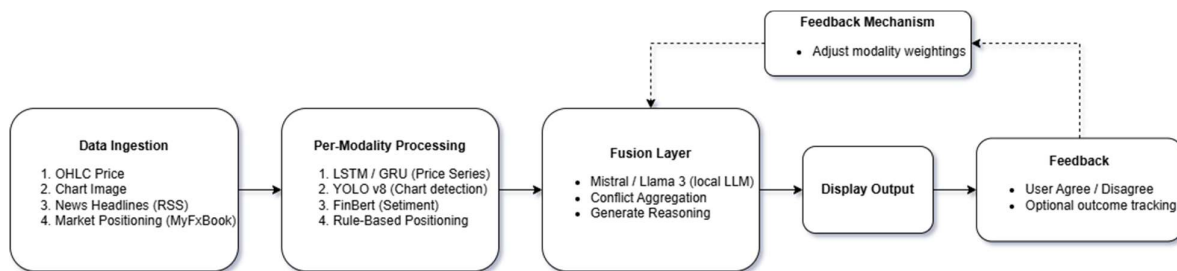


Figure 2: Multi-Modal Fusion System Architecture

3.5. Technologies and Methods

3.5.1. Language and Orchestration

Python. Pipeline orchestration handled through a custom Python pipeline class with sequential stage execution. No agent framework required given the controlled, unidirectional data flow.

3.5.2. Dashboard

Streamlit for the frontend interface. Chosen for development speed, Python-native integration and suitability for academic demonstration. Provides interactive asset selection, analysis trigger, structured output display and feedback collection.

3.5.3. Sentiment model candidates

- ProsusAI/finbert (HuggingFace) - BERT fine-tuned on financial news, three-class output (positive, negative, neutral)
- yiyanghkust/finbert-tone - fine-tuned on analyst tone, complementary signal to news sentiment
- Selection criteria: F1 score on a held-out set of labelled financial news headlines

3.5.4. Chart/Image model candidates

- foduucm/stockmarket-pattern-detection-yolov8 (HuggingFace) - YOLOv8-based pattern detection across chart regions

- Candlestick pattern classifier (HuggingFace) - classification model outputting per-pattern probabilities
- Selection criteria: accuracy and confidence calibration on a set of manually labelled chart images representing known patterns

3.5.5. Time-series model candidates

- Rogendo/Forex-LSTM-Predictor-Collection (HuggingFace) - collection of pre-trained LSTM models per currency pair and timeframe
- Custom LSTM/GRU trained on yfinance OHLCV data for the target asset
- Selection criteria: directional accuracy (whether predicted direction matches actual next-period movement) on held-out historical data

3.5.6. Fusion/reasoning model candidates

- Llama 3 8B (Q4_0 quantized) via Ollama locally
- Mistral 7B (Q4_K_M quantized) via Ollama locally
- API-hosted proprietary models (eg. GPT, Claude) were excluded to avoid recurring API costs and external service dependencies
- Selection criteria: coherence and consistency of structured output across a fixed set of test inputs with known modality signals, evaluated manually and against a scoring rubric

3.5.7. Data sources

- yfinance: price and OHLCV data. Chosen over broker APIs such as OANDA for accessibility, no authentication required and suitability for historical and near-real-time data without live trading requirements
- RSS feeds (FXStreet, ForexLive): financial news. Chosen over NewsAPI for no rate limits and forex-specific coverage. Web scraping avoided on ethical and legal grounds
- MyFxBook public community outlook API: retail positioning data aggregated across brokers, providing broader market representation than single-broker sources

3.6. Work Plan

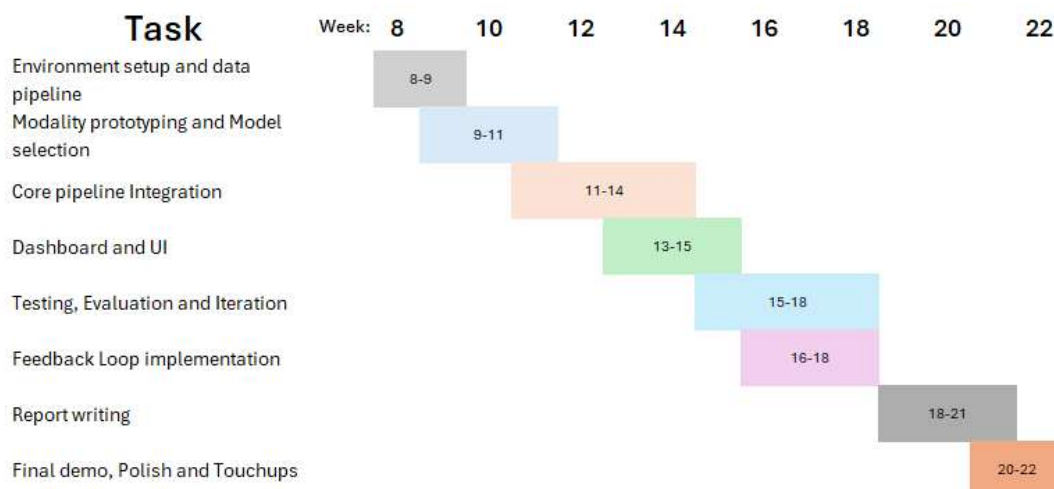


Figure 3: Gantt chart of project development timeline

Detailed Phase Descriptions

- Weeks 8-9 (Environment Setup and Data Pipeline): Configure development environment including Python dependencies, Ollama for local LLM hosting and HuggingFace model access. Establish data ingestion for yfinance price data, RSS news feeds and MyFxBook positioning API.
- Weeks 9-11 (Modality Prototyping and Model Selection): Implement and compare candidate models per role. Sentiment: ProsusAI/finbert vs yiyanghkust/finbert-tone. Chart: foduucm YOLOv8 vs candlestick classifier. Time-series: custom GRU vs custom LSTM vs pretrained Rogendo LSTM. Fusion: Mistral 7B vs Llama 3 8B. Select best performer per role based on defined criteria.
- Weeks 11-14 (Core Pipeline Integration): Wire selected models into a sequential pipeline. Implement conflict level computation, fusion prompt construction and structured JSON output validation. Integrate asset-aware sentiment filtering and live data ingestion.
- Weeks 13-15 (Dashboard and UI): Build Streamlit dashboard with asset selector, analysis trigger, structured output display and user feedback collection interface.
- Weeks 15-18 (Testing, Evaluation and Iteration): Run baseline comparisons against price-only, text-only and naive LLM configurations. Assess coherence, consistency and conflict accuracy across test cases. Collect peer feedback and iterate.
- Weeks 16-18 (Feedback Loop Implementation): Implement weight persistence across sessions, confidence score multipliers and outcome tracking logic.
- Weeks 18-21 (Report Writing): Draft and refine project report covering implementation, evaluation results and analysis.
- Weeks 20-22 (Final Demo and Polish): Record final demo video, complete progress logs, polish codebase and prepare final submission.

Contingency is built into the overlapping phases between Weeks 15-18. If the MyFxBook API proves inaccessible without authentication, the positioning modality falls back to a manually maintained long/short ratio dataset sourced from broker public disclosures. If the chart pattern model produces unreliable outputs after testing, the image modality falls back to a ResNet-50 feature extractor with a lightweight classification head trained on a labelled candlestick dataset. If selected LLMs fail to run reliably in a local environment due to resource or latency constraints, a smaller model or simplified rule-based fusion approach will be used instead. In cases where data sources are unavailable or inconsistent, the pipeline will continue with partial modality inputs using cached or most recent valid data. If integration issues arise, individual components will be tested using mocked outputs and the system scope reduced to core modalities to ensure completion.

3.7. Evaluation Plan

3.7.1. Level 1: Per-modality model selection

Before pipeline assembly, each model role is evaluated independently against defined criteria. Sentiment models are compared on F1 score across positive, negative and neutral classes on a labelled financial news dataset. Chart models are compared on pattern classification accuracy on a manually labelled set of chart images. Time-series models are compared on directional accuracy on held-out historical price data. Fusion models are compared on output coherence and consistency

assessed against a fixed scoring rubric applied to a set of standardised test inputs. The winning model per role is selected based on these results, with selection decisions documented and justified.

3.7.2. Level 2: System-level evaluation

The full pipeline is evaluated against three baselines:

- Baseline 1 (price-only): structured data modality alone, equivalent to traditional technical indicator analysis
- Baseline 2 (text-only): sentiment modality alone, equivalent to a pure news-driven approach
- Baseline 3 (naive LLM): all raw data fed into the fusion LLM without specialist model processing, demonstrating the value of the structured pipeline over an unorchestrated approach

Evaluation metrics at system level:

- Coherence: does the reasoning summary make sense given the per-modality signals? Assessed manually against a rubric across a fixed set of test cases
- Consistency: do similar inputs produce stable outputs? Assessed by running identical inputs multiple times and measuring output variance
- Conflict accuracy: does the conflict level correctly reflect the degree of disagreement between modality signals? Assessed against manually labelled test cases where signal agreement or disagreement is known

3.7.3. Level 3: Feedback loop evaluation

To test hypothesis H3 (that feedback improves consistency over time), the system is run over a sequence of analyses with simulated feedback applied. Output consistency is measured before and after weighting adjustments to assess whether the feedback mechanism produces meaningful improvement.

This evaluation design directly addresses the three project hypotheses:

- H1 (multi-modal outperforms single-modality): addressed by Level 2 baseline comparisons
- H2 (explicit conflict modelling improves interpretability): addressed by Level 2 coherence assessment, conflict-scoring verification and qualitative analysis of the generated reasoning.
- H3 (feedback improves consistency): addressed by Level 3 feedback loop evaluation

4. Implementation

The candidate technologies proposed in §3.5 were evaluated through feature prototypes before implementation. The resulting selections: ProsusAI/FinBERT, YOLOv8, a custom GRU and Mistral 7B; form the implemented pipeline described in this chapter. Representative comparison outputs are provided in Appendix A.1~A.4, with the supporting notebooks included in *‘feature_prototype/’* directory.

4.1. Architecture and Orchestration

The system is implemented as a Streamlit application split into four application layers: *‘infrastructure/’*, containing configuration constants and the PostgreSQL connection logic; *‘core/’*, containing the orchestrator, fusion and feedback logic; *‘modalities/’*, containing one module per data source; and *‘ui/’*, containing the Streamlit views, HTML/CSS component builders and shared UI constants.

main.py wires these together and is kept deliberately thin, holding only session-state management and the run trigger. A fifth top-level directory, *‘models/’*, holds trained artifacts rather than application code and is read directly by *‘modalities/timeseries.py’* at runtime, not imported as a package.

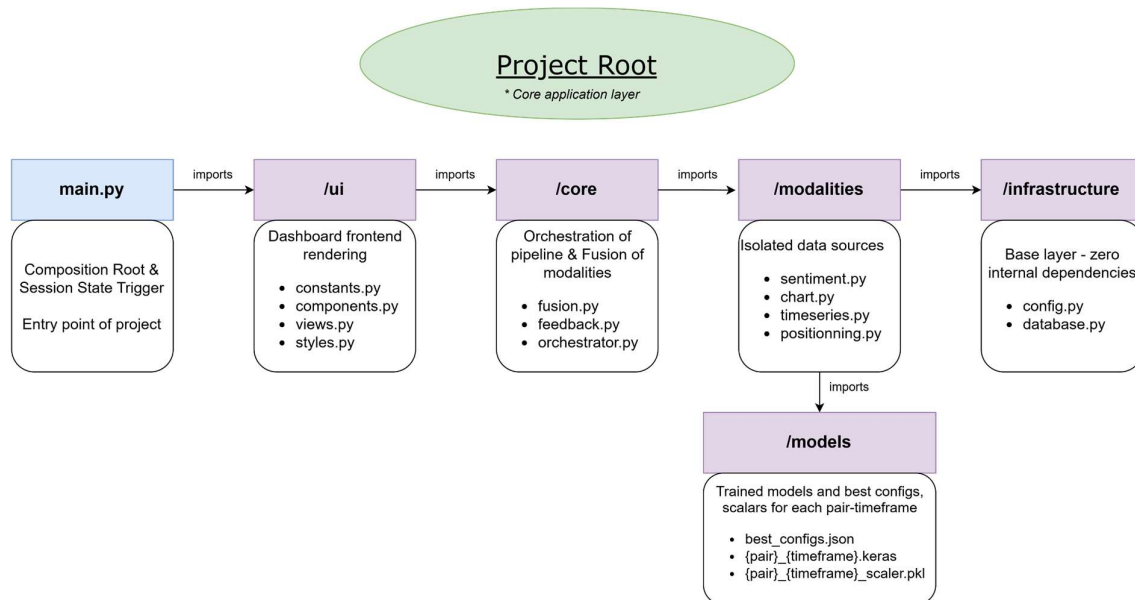


Figure 4: High-level project architecture illustrating the application layers and one-way dependency flow.

Coordination is handled by a single Orchestrator class rather than a task queue or agent framework, matching the sequential, single-user execution model the dashboard requires. The orchestrator is instantiated once per Streamlit session via *st.cache_resource*, so the two heavy models, FinBERT and YOLOv8, are loaded into memory only on first use; reloading both on every run added 30~60 seconds of dead time during testing. The GRU model is the exception to this caching: because a different model/scaler pair exists for every pair-timeframe combination, *load_gru()* runs fresh on every call rather than being preloaded at orchestrator init. Each of the four modality calls is wrapped

in a try/except at the orchestrator level, so a failure in one modality (eg. YOLO error) falls back to a neutral signal for that modality rather than crashing the run.

Each modality module exposes exactly one entry point, `get_sentiment_signal()`, `get_chart_signal()` and so on; that the orchestrator calls without needing to know anything about what happens inside it. This keeps a modality replaceable in isolation: swapping the sentiment model or rewriting the positioning rule only touches that one file, with no change required to orchestration code.

4.2. Modality 1: News/Sentiment

News sentiment is sourced from RSS feeds (FXStreet, ForexLive), filtered by a currency-keyword dictionary that is checked against both legs of the requested pair, so a USD-only headline is still captured for EURUSD even though “EUR” never appears in the text. ProsusAI/finbert classifies each matched headline into positive/negative/neutral with a confidence score.

The main technical piece of this modality is converting headline sentiment into a pair-relative signal. `detect_currency_focus()` counts keyword occurrences for each side of the pair to decide which currency a headline is actually about. `apply_currency_adjustment()` then maps FinBERT's raw label onto a direction using a per-pair effect table, positive USD sentiment becomes bearish for EURUSD but bullish for USDJPY, for example. Headlines with an ambiguous currency focus or a neutral label contribute no signal at all rather than being averaged in as zero.

```
def detect_currency_focus(headline: str, pair: str) -> str:
    """Detects which currency in the pair the headline focus on/mainly affects.
    base_currency = pair[:3]
    quote_currency = pair[3:]

    base_keywords = CURRENCY_KEYWORDS.get(base_currency, [])
    quote_keywords = CURRENCY_KEYWORDS.get(quote_currency, [])
    headline_lower = headline.lower()

    # Count keyword matches for each currency (simple frequency heuristic)
    base_hits = sum(1 for kw in base_keywords if kw.lower() in headline_lower)
    quote_hits = sum(1 for kw in quote_keywords if kw.lower() in headline_lower)

    # More keyword hit count indicates the focus
    if base_hits > quote_hits:
        return base_currency
    elif quote_hits > base_hits:
        return quote_currency
    else:
        return "neutral" # if tied
```

Listing 1: Detecting the headline's primary currency focus using keyword matching

```
def apply_currency_adjustment(classified: list[dict], pair: str) -> list[dict]:
    """Maps raw FinBERT sentiment to a directional signal for the specific pair.
    pair_effect = PAIR_CURRENCY_EFFECT.get(pair, {})
    adjusted = []

    for item in classified:
        label = item["label"]
        currency_focus = detect_currency_focus(item["headline"], pair)

        if label == "neutral" or currency_focus == "neutral":
            # Skip neutral sentiments as give no signal
            signal = 0
        else:
            # Map currency to directional effect on pair (+1, -1)
            direction = pair_effect.get(currency_focus, 0)
            # Positive sentiment follows direction and negative will flip it
            if label == "positive":
                signal = direction
            elif label == "negative":
                signal = -direction
            else:
                signal = 0

        adjusted.append({"item": item, "signal": signal})

    return adjusted
```

Listing 2: Converting FinBERT sentiment into pair-relative trading signals

Per-headline signals are combined with a confidence-weighted average (`aggregate_signal()`) rather than a plain majority vote, so a handful of high-confidence headlines can outweigh a larger number of uncertain ones. A ± 0.1 band around zero suppresses weak aggregate signals instead of forcing a direction on thin evidence.

The confidence value this modality reports is not FinBERT's classification confidence, it's the strength of directional agreement across headlines, `abs(weighted_avg)`. The distinction matters: ten headlines FinBERT scores at 90% neutral would report near-zero confidence here, correctly reflecting that there's no directional signal, even though the underlying model was very sure about its (neutral) reading. Passing FinBERT's confidence straight through would overstate conviction on

exactly the headlines carrying no direction, distorting both the conflict score and this modality's feedback-driven weight downstream.

```
INFO:httpx:HTTP Request: HEAD https://huggingface.co/ProsusAI/finbert/resolve/refs%2Fpr%2F29/model.safetensors "HTTP/1.1 302 Found"
INFO: __main__: Fetched 21 filtered headlines for EURUSD.
INFO: __main__: Sentiment signal for EURUSD: {'signal': 1, 'confidence': 0.1002, 'distribution': {'positive': 6, 'negative': 12, 'neutral': 3}}
{'signal': 1, 'confidence': 0.1002, 'distribution': {'positive': 6, 'negative': 12, 'neutral': 3}}
```

Figure 5: Sentiment modality output for a live EUR/USD run, including the aggregated signal, confidence and sentiment distribution

4.3. Modality 2: Chart Image Pattern Detection

Chart images are generated on demand from live yfinance OHLC data using mplfinance, so the same code path produces a chart for any supported pair/timeframe combination. The image is passed to a YOLOv8 pattern-detection model (foducom/stockmarket-pattern-detection-yolov8) trained to detect six structural patterns: head-and-shoulders top/bottom, double top/bottom, triangle and trendline.

`run_yolo_inference()` keeps the highest-confidence detection when multiple patterns appear on the same chart and returns a neutral, zero-confidence result if nothing is detected. It maps to a directional signal: head-and-shoulders top and M-head are bearish, head-and-shoulders bottom and W-bottom are bullish, while triangle and trendline detections are neutral.



Figure 6: YOLO output highlighting the highest-confidence chart pattern detected on a live EUR/USD H1 chart

```

def run_yolo_inference(model: YOLO, chart_path: str, save_annotated: bool = False) -> dict:
    """Runs YOLO inference on chart image...
    results = model(chart_path, verbose=False)
    boxes = results[0].boxes

    if boxes and len(boxes) > 0:
        # Take highest confidence detection score
        confidences = boxes.conf.tolist()
        best_idx = confidences.index(max(confidences))
        cls_idx = int(boxes.cls[best_idx].item())
        conf = confidences[best_idx]
        label = YOLO_CLASSES[cls_idx] if cls_idx < len(YOLO_CLASSES) else "none"
    else:
        logger.info("YOLO detected no patterns in chart.")
        label = "none"
        conf = 0.0

    # Convert detected pattern into a directional trading signal
    signal = YOLO_SIGNAL_MAP.get(label, 0)

    logger.info(f"YOLO result: {label} | signal={signal:+d} | conf={conf:.3f}")

    # Save annotated chart showing only the highest-confidence detection
    if save_annotated and boxes and len(boxes) > 0:
        best_result = copy.deepcopy(results[0])
        best_result.boxes = best_result.boxes[best_idx:best_idx + 1]
        best_result.save(filename="chart_output_annotated.png")
    return {
        "signal": signal,
        "confidence": round(conf, 4),
        "pattern": label
    }

```

Listing 3: run_yolo_inference() selects the highest-confidence detection and maps it to a trading signal

4.4. Modality 3: Price/Time-Series

Directional prediction uses a dual-output GRU, one head regressing next-candle price change, one head classifying direction, trained separately for each pair/timeframe combination rather than as a single general model. The tuning process (*GRU_tuning.py*) sweeps 16 configurations per combination: two feature sets (raw OHLC vs OHLC plus RSI/MACD/EMA20/ATR), two lookback windows (15 vs 30 candles), two dropout rates (0.2 vs 0.3) and two GRU layer sizes (32→16 vs. 64→32). Each trained on an 80/20 chronological split with early stopping on validation loss. This runs across all seven supported pairs on both the 1-hour and daily timeframe, 14 pair/timeframe combinations, 224 trained models in total.

The winning configuration per pair/timeframe, together with its validation accuracy and saved model/scaler paths, is written to *'models/best_configs.json'*.

	asset	timeframe	accuracy	lookback	dropout	gru1	gru2
3	GBP	USD_1d	84.51	30	0.2	64	32
11	USD	CHF_1d	83.43	15	0.2	32	16
1	EUR	USD_1d	81.09	15	0.2	32	16
5	AUD	USD_1d	80.98	30	0.3	64	32
13	USD	CAD_1d	80.31	15	0.2	32	16
7	NZD	USD_1d	80.20	30	0.2	64	32
0	EUR	USD_1h	57.82	30	0.2	32	16
12	USD	CAD_1h	55.39	15	0.2	64	32
8	USD	JPY_1h	55.03	30	0.2	64	32
9	USD	JPY_1d	53.80	15	0.3	32	16
10	USD	CHF_1h	53.54	15	0.3	64	32
4	AUD	USD_1h	53.26	15	0.3	32	16
2	GBP	USD_1h	53.21	15	0.2	64	32
6	NZD	USD_1h	52.91	15	0.3	64	32

Table 2: Best-performing GRU configuration for each currency pair and timeframe

At inference time, *timeseries.py* reads *best_configs.json* to determine the feature set, lookback and saved model/scaler for the requested pair and timeframe (*load_best_config()*, *load_gru()*) rather than assuming a single fixed configuration. This also means indicator computation is conditional: *compute_indicators()* only runs when the winning config actually needs RSI/MACD/EMA20/ATR, skipping that step entirely for the five winning configurations that use plain OHLC. *run_gru_inference()* applies the pre-fitted *RobustScaler*, never refitted at inference, to the latest window of live data and thresholds the direction-probability output at 0.5 to produce a ± 1 signal, with confidence taken as distance from that threshold: a direction probability of 0.73 becomes a bullish signal at 73% confidence, while 0.38 becomes a bearish signal at 62% confidence (**1 - 0.38**), so the reported figure always reads as “how sure the model is in the direction it picked” rather than the raw up-class probability.

```
def load_best_config(pair: str, timeframe: str) -> dict:
    """Reads best_configs.json (produced by the feature prototype training notebook)"""
    if not os.path.exists(GRU_BEST_CONFIGS_PATH):
        raise FileNotFoundError(
            f"best_configs.json not found at {GRU_BEST_CONFIGS_PATH}. "
            f"Run the training notebook to generate it."
        )

    with open(GRU_BEST_CONFIGS_PATH, "r") as f:
        best_configs = json.load(f)

    key = f"{pair}_{timeframe}" # config key format eg. EURUSD_1h
    if key not in best_configs:
        raise KeyError(
            f"No entry for '{key}' in {GRU_BEST_CONFIGS_PATH}. "
            f"Check that training completed for this pair and timeframe."
        )

    cfg = best_configs[key]
    logger.info(f"Best config for {key}: features={cfg['features']} lookback={cfg['lookback']}")
    return cfg
```

Listing 4: *load_best_config()* loads best GRU config for the requested currency pair and timeframe

```
def run_gru_inference(model: Model, scaler: df: pd.DataFrame, features: list, lookback: int) -> dict:
    """Runs GRU inference on the most recent 'lookback' rows of fresh data. ..."""
    missing = [f for f in features if f not in df.columns]
    if missing:
        raise ValueError(f"DataFrame missing features required by model: {missing}")

    data = df[features].values
    # Scale using the training scaler (never refit on inference data)
    data_scaled = scaler.transform(data)

    if len(data_scaled) < lookback:
        raise ValueError(
            f"Not enough rows after indicator warmup: have {len(data_scaled)}, need {lookback}. "
            f"Try a longer fetch period or check indicator computation."
        )

    X = data_scaled[-lookback:].reshape(1, lookback, len(features)) # use latest lookback window
    preds = model.predict(X, verbose=0)
    dir_prob = float(preds[1].flatten()[0])

    # 0.5 porbability threshold for binary direction prediction
    signal = 1 if dir_prob > 0.5 else -1
    confidence = round(dir_prob if dir_prob > 0.5 else 1 - dir_prob, 4)

    logger.info(f"GRU inference: dir_prob={dir_prob:.4f} signal={signal:+d} conf={confidence:.4f}")
    return {
        "signal": signal,
        "confidence": confidence,
        "direction_probability": round(dir_prob, 4)
    }
```

Listing 5: *run_gru_inference()* applies the trained GRU model to produce a directional signal and confidence score

4.5. Modality 4: Market Positioning

Positioning uses a live, session-based integration with the MyFxBook community outlook API (*login()* / *fetch_community_outlook()* / *logout()*), authenticating with credentials read from environment variables and always logging out in a finally block regardless of whether the request succeeded. *apply_contrarian_rule()* converts the returned long/short percentage split into a signal using a fixed 60% threshold: if 60% or more of retail traders are net short, the signal is bullish and vice versa. Reflecting the well-documented tendency of retail sentiment to sit on the wrong side of extreme moves. Both the threshold and the confidence score (the short or long percentage divided by 100) remain the same fixed, linear form used at the prototype stage, kept for interpretability given the rule-based design.

Every request (login, outlook fetch, logout) carries a 10-second timeout, so an unresponsive MyFxBook endpoint can't stall the pipeline. If the API call fails, times out, the pair isn't present in the response, or neither side crosses the threshold, the modality returns a neutral fallback rather than forcing a direction.

```
def apply_contrarian_rule(pair_data: dict) -> dict:
    """Converts retail positioning into a contrarian trading signal. ...
    short_pct = pair_data.get("shortPercentage", 50)
    long_pct = pair_data.get("longPercentage", 50)

    # Majority retail traders are short -> contrarian bullish
    if short_pct >= CONTRARIAN_THRESHOLD:
        signal = 1
        confidence = round(short_pct / 100, 4)
        note = f"Retail {short_pct}% short - contrarian bullish"
    # Majority retail traders are long -> contrarian bearish
    elif long_pct >= CONTRARIAN_THRESHOLD:
        signal = -1
        confidence = round(long_pct / 100, 4)
        note = f"Retail {long_pct}% long - contrarian bearish"
    # No positioning that hits/exceeds threshold detected -> neutral
    else:
        signal = 0
        confidence = 0.5
        note = f"No extreme positioning ({long_pct}% long / {short_pct}% short) - neutral"

    logger.info(f"Positioning signal: {note}")
    return {
        "signal": signal,
        "confidence": confidence,
        "long_pct": long_pct,
        "short_pct": short_pct,
        "note": note
    }
```

Listing 6: *apply_contrarian_rule()* converts retail positioning into a contrarian trading signal

```
INFO: _main_: MyFxBook login successful.
INFO: _main_: Fetched community outlook for 186 symbols.
INFO: _main_: Positioning signal: Retail 62% long - contrarian bearish
INFO: _main_: Positioning signal for EURUSD: {'signal': -1, 'confidence': 0.62, 'long_pct': 62, 'short_pct': 38, 'note': 'Retail 62% long - contrarian bearish'}
INFO: _main_: MyFxBook logout successful.
{'signal': -1, 'confidence': 0.62, 'long_pct': 62, 'short_pct': 38, 'note': 'Retail 62% long - contrarian bearish'}
```

Figure 7: Example positioning modality output showing the resulting contrarian trading signal using live data

4.6. Fusion Layer

The fusion layer combines the four weighted modality outputs into a single structured decision using a locally hosted LLM (Mistral 7B via Ollama). Two things happen before the model is called: a conflict score and the prompt itself.

`compute_conflict()` sums the confidence of every bullish-signalling modality separately from every bearish-signalling one, then measures how imbalanced that split is: $|\text{bull} - \text{bear}| / (\text{bull} + \text{bear})$. A high imbalance means the modalities that do hold a directional opinion are pointing the same way, so conflict reads as low even when one of the four is neutral. This is a deliberate move away from counting +1/-1/0 signals directly, since a plain count would treat a modality reporting 90% confidence identically to one reporting 51%, understating genuine agreement between the strongest signals.

```
def compute_conflict(modality_outputs: dict) -> tuple[str, str, float]:
    """Computes conflict level using confidence-weighted signal imbalance/agreement. ...
    # Sum confidence where models say its bullish
    weighted_bull = sum(
        m["confidence"] for m in modality_outputs.values()
        if m["signal"] == 1
    )
    # Sum confidence where models say its bearish
    weighted_bear = sum(
        m["confidence"] for m in modality_outputs.values()
        if m["signal"] == -1
    )
    total = weighted_bull + weighted_bear # combined total
    # If no directional signal (0), means neutral
    # * imbalance is reported as 0.0 here, not a "measured" agreement, just nothing to disagree on
    if total == 0:
        return "low", "all modalities neutral", 0.0

    # Calculate imbalance (how one-sided signals are) using basic normalized difference
    imbalance = abs(weighted_bull - weighted_bear) / total

    # Higher imbalance -> stronger agreement -> lower conflict
    # Lower imbalance -> greater disagreement -> higher conflict
    if imbalance > CONFLICT_LOW_THRESHOLD:
        conflict_level = "low"
        conflict_note = f"strong directional agreement (imbalance={imbalance:.2f})"
    elif imbalance > CONFLICT_MEDIUM_THRESHOLD:
        conflict_level = "medium"
        conflict_note = f"partial agreement (imbalance={imbalance:.2f})"
    else:
        conflict_level = "high"
        conflict_note = f"significant disagreement (imbalance={imbalance:.2f})"

    logger.info(f"Conflict: {conflict_level} | weighted_bull={weighted_bull:.3f} weighted_bear={weighted_bear:.3f} imbalance={imbalance:.3f}")
    return conflict_level, conflict_note, imbalance
```

Listing 7: `compute_conflict()` computes confidence-weighted agreement between modality signals

`build_fusion_prompt()` assembles the four modality outputs, their signals, raw and weight-adjusted confidence scores (where applicable) and modality-specific context. Such as the detected chart pattern or retail long/short split, into a structured prompt. The prompt includes the pre-computed conflict score and instructs the model to return a fixed JSON schema (stance, confidence, conflict level, a one-line note per modality and a reasoning string), with explicit instructions not to hedge when conflict is already low. The response is parsed with `json_repair` rather than manual string matching, tolerating markdown-fenced or truncated responses; `validate_output()` then checks the result against the required schema before acceptance. `call_llm()` retries up to three times on a malformed or invalid response before falling back to a fixed neutral default, so a single bad generation can't take down the run.

```
INFO: _main_: conflict: high | weighted_bull:1.340 weighted_bear:0.704 imbalance:0.256
INFO: _main_: _llm call attempt 1/3...
INFO: httpx.HTTP Request: POST http://127.0.0.1:11034/v1/chat "HTTP/1.1 200 OK"
INFO: _main_: _llm output valid.
INFO: _main_: _fusion output for EURUSD 1h: stance-bullish confidence=0.62 conflict=high
{
  "stance": "bullish",
  "confidence": 0.62,
  "conflict_level": "high",
  "signals": {
    "news": "News sentiment indicates a slight positive distribution with a confidence of 0.72",
    "chart": "The detected chart pattern is a head and shoulders top, which could be bearish but carries reduced influence for this pair",
    "timeseries": "Time-series analysis suggests a bearish trend with a low confidence of 0.44",
    "positioning": "Retail positioning shows a strong bullish bias with 88% long and 62% short positions"
  },
  "reasoning": "Despite the bearish signals from chart pattern and time-series analysis, the overall stance is bullish due to the strong retail positioning and positive news sentiment. However, it's important to note that there is a high conflict level, which may indicate increased market volatility."
}
```

Figure 8: Example of validated and structured trading decision produced by the fusion LLM

```
def build_fusion_prompt(pair: str, timeframe: str, modality_outputs: dict, conflict_level: str, conflict_note: str) -> str:
    """Builds structured prompt for the fusion LLM. ...

    # All the modalities + current date
    news = modality_outputs["news"]
    chart = modality_outputs["chart"]
    ts = modality_outputs["timeseries"]
    pos = modality_outputs["positioning"]
    today = date.today().isoformat()

    # Nested helper to format confidence display
    def conf_line(data: dict) -> str:
        # Show weight-adjusted confidence if weighting multiplier applied (!= 0)
        # * else default to 1.0 if no feedback history
        w = data.get("weight", 1.0)

        # Raw confidence from original model output (before any weighting)
        raw = data.get("raw_confidence", data.get("confidence", 0.0))

        # Effective confidence AFTER weighting, what Fusion layer will actually use
        eff = data.get("confidence", 0.0)

        # If weight ~1.0 means no adjustment applied, just show final confidence
        if abs(w - 1.0) < 0.001:
            return f"confidence: {eff}"

        # But if there is weightings applied, show all the components for transparency
        return (
            f"raw_confidence: {raw} | weight: {w} | "
            f"effective_confidence: {eff} "
            f"(weight reflects historical reliability for this pair)"
        )

    # Improved prompt from feature prototype
    # * Initially it was hedging response and acknowledging 'no conflicts' which is noise in reasoning
    # old feature prototype prompt - "<2-3 sentences synthesising overall stance, acknowledging any conflicts>"
    # * In iteration 2, we increase the prompt to "4-5" instead of "2-3" sentence as UAT suggest it was too brief
    prompt = f"""...

    # Use text wrap to cut away the whitespaces in the prompt
    # * i added those whitespaces so me and viewers can read the code better
    return textwrap.dedent(prompt).strip()
```

Listing 8: build_fusion_prompt() constructs the structured prompt for the fusion LLM

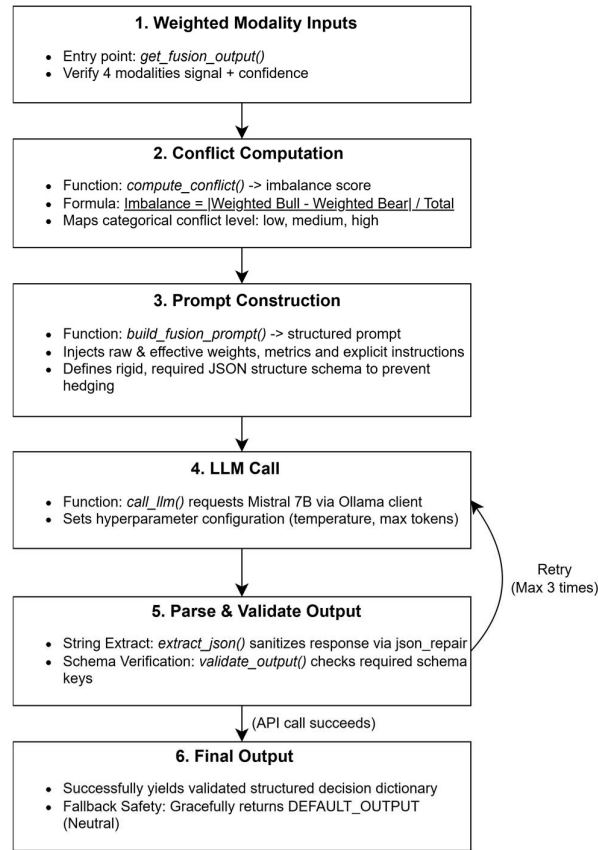


Figure 9: Fusion layer's sequence of operation, from weighted modality inputs to the final validated trading decision

4.7. Feedback Mechanism and Persistence

Every completed run is written to a `feedback_log` table in PostgreSQL (`insert_analysis()`), storing the full modality payload and reasoning text as JSONB alongside the stance/confidence/conflict summary, with a composite index on (pair, run_at) supporting the history panel's per-pair lookups. Postgres was chosen specifically for JSONB, which lets the nested per-modality payload be stored and later queried without flattening it into separate columns. `init_db()` creates both tables with CREATE TABLE IF NOT EXISTS, so startup is idempotent across repeated restarts rather than failing on the second run.

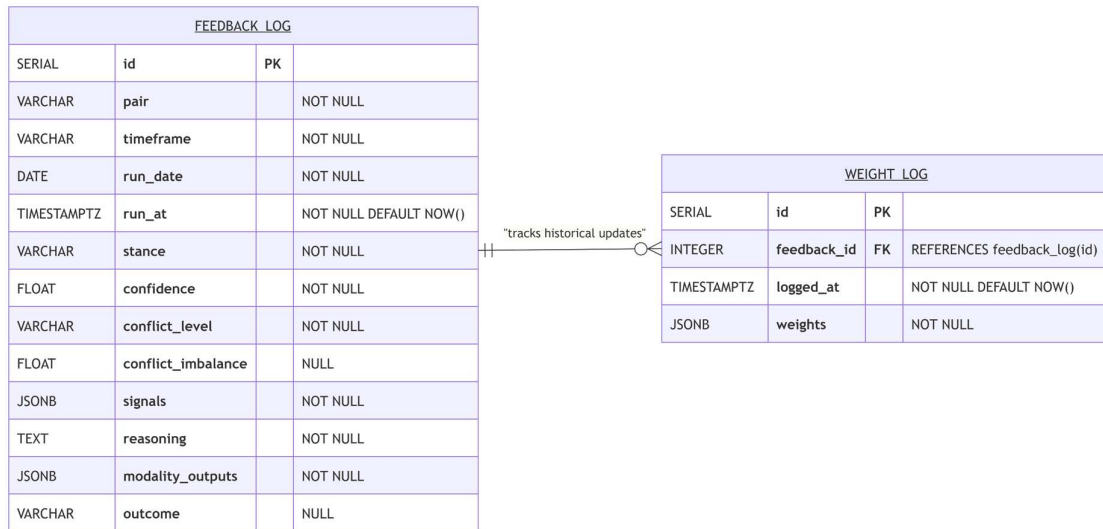


Figure 10: PostgreSQL schema showing the `feedback_log` and `weight_log` tables

When a user marks a completed analysis as correct, incorrect, or uncertain, `process_outcome_feedback()` and `compute_weight_update()` decide whether and how each modality's per-pair weight multiplier should change. The rule distinguishes four cases: a modality that agreed with a correct stance is rewarded (+0.1, capped at a ceiling of 2.0); a modality that agreed with an incorrect stance is penalised (-0.1, floored at 0.1); a modality that disagreed with an incorrect stance is rewarded, since it was right to dissent from the consensus; and a modality that disagreed with a correct stance is left unchanged, since penalising a cautious or contrarian signal purely for being outvoted would discourage genuine disagreement in future runs. An update is skipped entirely if the outcome is marked uncertain, or if every modality agreed on a stance that still turned out wrong, in that case there's no differential signal to attribute the miss to.

```

def compute_weight_update(weights: dict, modality_outputs: dict, outcome: str, stance: str) -> dict | None:
    """Computes updated weights after a user feedback event..."""
    if outcome == "uncertain":
        logger.info("Outcome uncertain - no weight update.")
        return None

    # Map stance string to signal int for comparison
    stance_signal = {"bullish": 1, "bearish": -1, "neutral": 0}.get(stance, 0)

    # Check if all non-neutral modalities agreed (since we dont care about uncertain neutrals)
    non_neutral = [
        m for m, d in modality_outputs.items()
        if d.get("signal", 0) != 0
    ]

    all_agreed = all(
        modality_outputs[m].get("signal", 0) == stance_signal
        for m in non_neutral
    ) if non_neutral else True # if empty list (neutrals), take it as all agree

    if outcome == "incorrect" and all_agreed:
        logger.info(
            "All modalities agreed on wrong outcome - no differential signal to attribute, skipping update."
        )
        return None

    updated = weights.copy()
    for mod in MODALITIES:
        sig = modality_outputs.get(mod, {}).get("signal", 0)
        agreed = (sig == stance_signal) and (stance_signal != 0) # whether modality supported the final stance

        # Reward correct contributors
        if outcome == "correct" and agreed:
            updated[mod] = round(min(weights[mod] + LEARNING_RATE, WEIGHT_CEILING), 3)
        # Penalise wrong decision contributors
        elif outcome == "incorrect" and agreed:
            updated[mod] = round(max(weights[mod] - LEARNING_RATE, WEIGHT_FLOOR), 3)
        # Reward dissenter (when modality disagrees with FUSION final decision that turns out to be wrong stance)
        elif outcome == "incorrect" and not agreed and sig != 0:
            updated[mod] = round(min(weights[mod] + LEARNING_RATE, WEIGHT_CEILING), 3)
        # correct + disagreed: no change (avoid punishing cautious signal)
        # neutral signal: no change regardless of outcome as no learning signal

    logger.info(f"Weight update: before={weights} after={updated}")
    return updated

```

Listing 9: `compute_weight_update()` updates per-pair modality weights from user feedback

Updated weights are stored as a new row in `weight_log` rather than overwritten in place, which is what makes the weight-trajectory chart on the dashboard possible, each feedback event produces one snapshot instead of losing prior history. `apply_weights()` then multiplies each modality's raw confidence by its current weight before every subsequent run, clamped to [0, 1], so a modality that has proven unreliable for a given pair carries less influence in later fusion prompts without any underlying model being retrained.

4.8. Dashboard UI and Execution Flow

The dashboard is a two-column Streamlit layout: controls, run history, the current weight panel on the left and the full analysis output on the right.

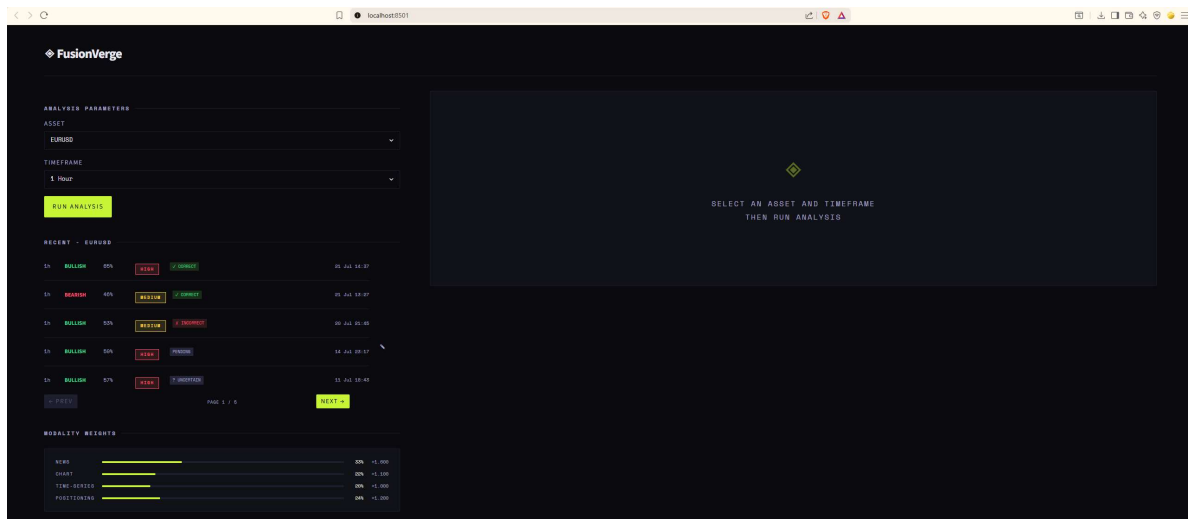


Figure 11: Dashboard before analysis execution, showing parameter selection, recent analysis history and the current modality weight panel

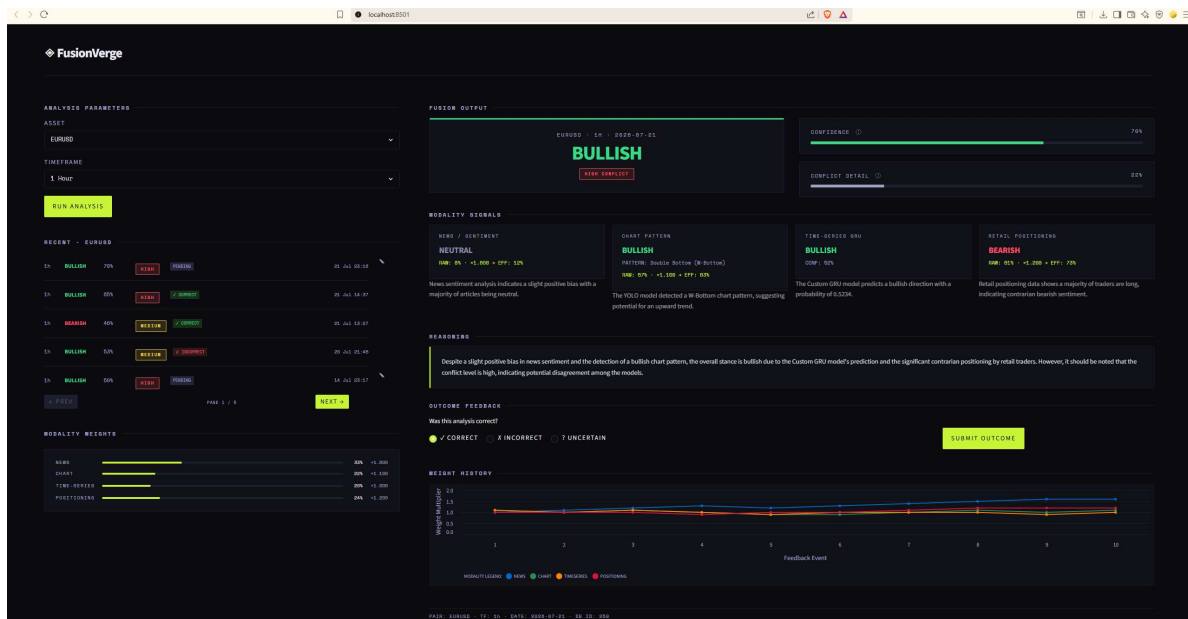


Figure 12: Dashboard after analysis execution, displaying the fused prediction, modality outputs, LLM reasoning, feedback controls and historical weight trajectory

Triggering “RUN ANALYSIS” does not execute the pipeline inside the button's own callback. *main.py* sets a running flag in session state and calls *st.rerun()*, which lets Streamlit flush a loading placeholder to the browser before the blocking *orchestrator.run()* call executes on the following pass, so the interface doesn't appear frozen for the 10~30 seconds a full run takes.

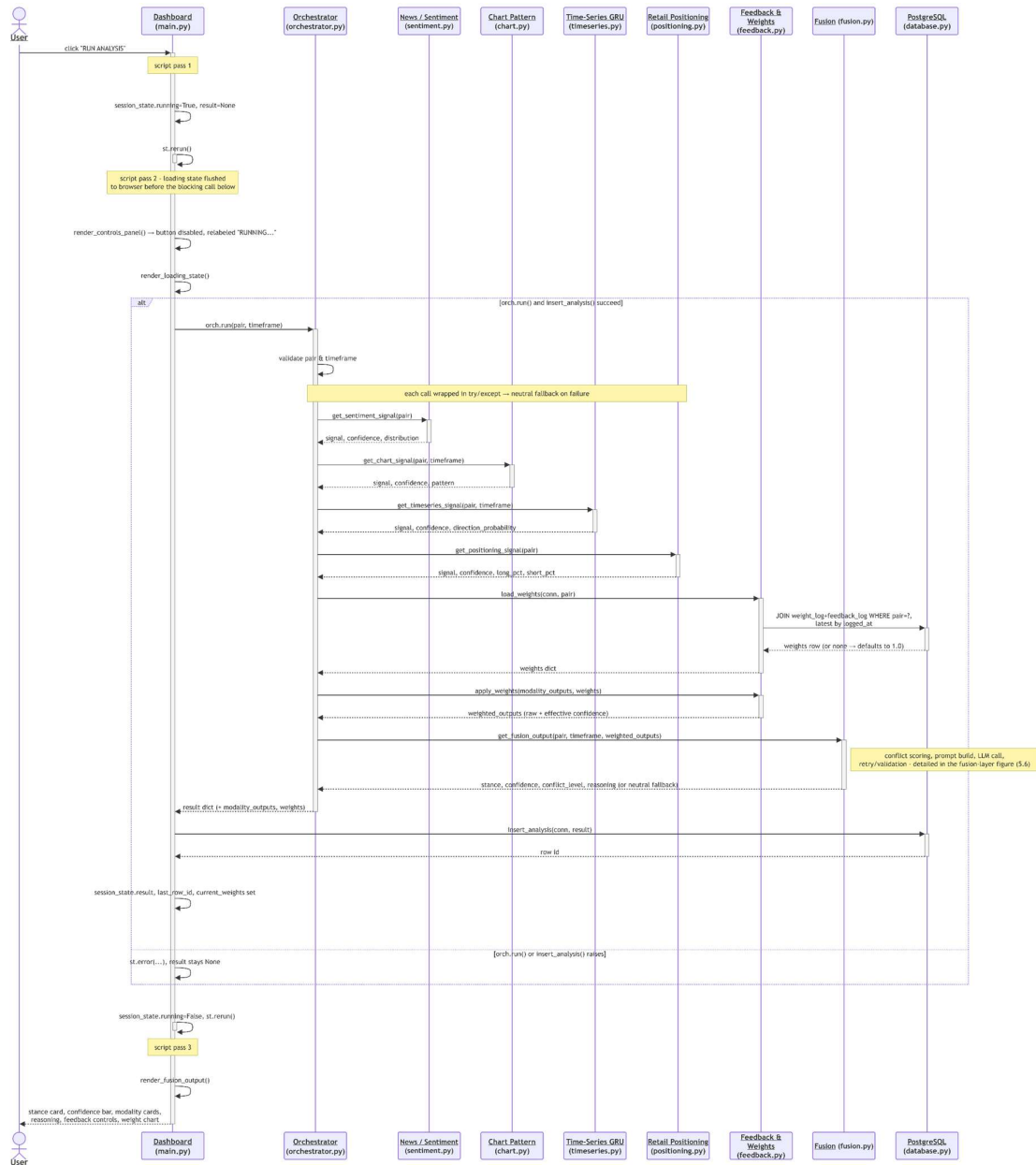


Figure 13: Sequence diagram showing the execution flow of a single analysis run, from dashboard interaction through modality execution, fusion, persistence and dashboard update

Once a run completes, `render_fusion_output()` displays the stance card, confidence and imbalance bar, and four modality signal cards. Each card shows the raw/weight-adjusted confidence. Below that sit the LLM's reasoning, outcome-feedback controls and a per-modality weight-trajectory chart, shown once at least one feedback event has been recorded for that pair.

Streamlit re-executes the entire script on every interaction rather than updating state in place, so anything that needs to persist across that rerun must live in `st.session_state` explicitly. `running` acts as a lock that disables the run button and relabels it "RUNNING..." for the duration of the pipeline call, preventing duplicate submissions; `result` and `last_row_id` hold the current analysis and its

database row ID so the feedback form can attach an outcome to the right record; and *feedback_submitted* disables the outcome form once a response has been recorded for that run.

Since an analysis's outcome isn't always recorded immediately, a pending row can be reopened later: *fetch_analysis_by_id()* reconstructs the full analysis output so the outcome-feedback controls have their original context, less the weight-trajectory chart, which reflects current weights rather than what applied at the time.

```
def run(self, pair: str, timeframe: str) -> dict:
    """Runs the full analysis pipeline for the given pair and timeframe...
    # Validate inputs first
    if pair not in SUPPORTED_ASSETS: ...
    if timeframe not in TIMEFRAME_CONFIGS: ...

    logger.info(f"Running analysis for {pair} {timeframe}...")

    # Modality 1: Sentiment (news)
    logger.info(f"[1/4] Sentiment...")
    try:
        news_output = get_sentiment_signal(pair, pipe=self.sentiment_pipe)
    except Exception as e: ...

    # Modality 2: Chart (YOLO pattern detection)
    logger.info(f"[2/4] Chart...")
    try:
        chart_output = get_chart_signal(pair, timeframe, yolo_model=self.yolo_model)
    except Exception as e: ...

    # Modality 3: Time-series (custom trained GRU)
    logger.info(f"[3/4] Time-series...")
    try:
        ts_output = get_timeseries_signal(pair, timeframe)
    except Exception as e: ...

    # Modality 4: Positioning (myfxbook API for contrarian rule)
    logger.info(f"[4/4] Positioning...")
    try:
        pos_output = get_positioning_signal(pair)
    except Exception as e: ...

    # Consolidate raw modality outputs
    modality_outputs = {
        "news": news_output,
        "chart": chart_output,
        "timeseries": ts_output,
        "positioning": pos_output,
    }

    # Apply weights to confidence BEFORE fusion layer
    # * per-pair weights with applied confidence multipliers
    # * default to 1.0 weight if no DB connection/history, as mentioned above
    # * more details in feedback.py and fusion.py (for application)
    if self.db_conn is not None: ...
    else: ...

    weighted_outputs = apply_weights(modality_outputs, weights)
    logger.info(f"Weights applied for {pair}: {weights}")

    logger.info("All modalities complete. Running fusion...")

    # Fusion layer (receives weighted confidence values)
    fusion_result = get_fusion_output(pair, timeframe, weighted_outputs)

    # Attach metadata
    # * for downstream use (dashboard/logging/feedback loop)
    result = { ...

    logger.info(...)
    return result
```

Listing 10: High-level folded view of *Orchestrator.run()*, showing the high-level execution flow coordinating the four modalities, weight application, fusion and result aggregation

5. Evaluation

5.1. Evaluation Strategy

Given the project's four independent modalities feeding a single fusion decision, evaluation was structured around the same layering as the system itself: each modality was tested in isolation before being tested as part of the whole. This follows the standard progression of unit testing, integration testing, system testing and user acceptance testing, with one adaptation specific to this project: several tests depend on live, rate-limited, or credential-gated external services (MyFxBook, Ollama, RSS feeds, trained model files), so tests touching these are conditionally skipped rather than mocked outright wherever the real behaviour of the dependency itself is what needs validating. This keeps the suite honest about what was actually exercised against live conditions versus simulated conditions, which matters for a project explicitly built around several unreliable third-party data sources.

Coverage was aimed at three things: whether individual pieces of novel logic (currency-adjustment mapping, confidence-weighted aggregation, confidence-weighted conflict scoring, weight-update rules) behave correctly under edge cases; whether the system degrades gracefully rather than crashing when a component fails; and whether the fusion layer's output is usable and trustworthy from a human's perspective, which unit tests cannot answer on their own, hence the questionnaire in §5.6.

5.2. Unit and Integration Testing

Module	Test count	Skip-gated on	Full results
sentiment.py	42	none	Appendix B.1
chart.py	21	none	Appendix B.2
timeseries.py	34	trained model files	Appendix B.3
positioning.py	25	MyFxBook credentials	Appendix B.4
feedback.py	40	none	Appendix B.5
fusion.py	52	Ollama (1 test)	Appendix B.6
orchestrator.py	31	Ollama + models (5)	Appendix B.7
database.py	21	live PostgreSQL	Appendix B.8
Total:	266		

Table 3: Summary of pytest results by module. Full test results are provided in Appendices B.1~B.8

Live-dependency gating varies by module: sentiment and chart touch live services (*RSS*, *yfinance*, *YOLO*, *FinBERT*) on every run with no skip guard; timeseries and orchestrator are conditionally skipped without trained model files; positioning is skip-gated on MyFxBook credentials; feedback runs fully mocked with no external dependency at all.

Sentiment and fusion carry the heaviest unit coverage, which tracks with them being the two most original pieces of logic in the system. *detect_currency_focus()* is tested across five pairs plus the ambiguous and unrelated-headline edge cases and *apply_currency_adjustment()* is tested across representative positive/negative base- and quote-currency scenarios. This is the part of the pipeline doing genuine domain reasoning rather than passing a model's output straight through, so it's tested accordingly.

The feedback and fusion test suites are entirely deterministic despite touching non-deterministic components. *compute_weight_update()* is tested against every branch its own docstring logic implies (Appendix C): agreeing-and-correct (boosted), agreeing-and-incorrect (penalised), disagreeing-and-correct (unchanged), unanimous-and-wrong (no update, nothing to attribute), all-uncertain (no update), plus floor/ceiling clamping and exact learning-rate-delta verification. *compute_conflict()* similarly tests each conflict band directly against the configured thresholds rather than magic numbers, so the tests stay valid if those thresholds are retuned later.

Graceful degradation is tested explicitly, not assumed. *test_fallback_on_bad_credentials* (positioning), the blank-image no-detection case (chart), and *test_returns_default_on_ollama_exception / test_returns_default_after_all_retries_exhausted* (fusion) all confirm the system returns a neutral or default result rather than propagating an exception; reinforcing the §4.1 design claim that single-modality failures should not terminate the overall run.

5.3. System-Level and Live Integration Testing

Beyond per-module tests, *test_orchestrator.py* includes a set of true system tests that only run against the live stack (requires_all: Ollama running, GRU models trained for EURUSD and GBPUSD). Three of these produce evidence for specific design claims made earlier in the report rather than just confirming the pipeline runs at all (full *pytest -v* output for this class is provided in Appendix B.1~B.8):

- **Model-reuse timing** (*test_model_reuse_across_calls*): asserts a second *run()* call on the same orchestrator instance is not slower than the first, directly testing the claim in §4.1 that caching FinBERT and YOLO at orchestrator *init* avoids the 30~60s reload cost on every analysis, with fuller multi-run timing evidence presented in §5.4.
- **Fusion consistency** (*test_consistency_five_consecutive_runs*): runs the same pair/timeframe five times and asserts no more than two distinct stances appear across the runs. This is the direct empirical answer to the LLM non-determinism limitation flagged during development, rather than leaving it as an acknowledged-but-unmeasured risk, it's a standing regression test; a larger 10-run version of this comparison with full summary statistics is presented in §5.4.
- **Full-pipeline output validity** (*test_eurusd_returns_valid_output*, *test_gbpusd_returns_valid_output*): confirms the complete metadata contract (stance,

confidence, conflict level, modality outputs, weights) holds end-to-end against live data, not just mocked fixtures.

`test_database.py` provides the remaining system-level coverage on the persistence layer: schema idempotency across restarts, JSONB round-tripping of the nested modality payload and correct ordering/filtering/limiting of the history query. The three areas most likely to silently corrupt data without a test catching it.

5.4. Hypothesis Testing (H1-H3)

The project's evaluation plan (§3.7) sets out three hypotheses the architecture was built to test. A multi-modal fused signal should outperform a single, unorchestrated language model (H1). Explicit conflict modelling should improve the interpretability of the fusion output (H2) and the feedback loop should improve output consistency over time (H3). The consistency testing in §5.3 already provides part of this evidence; this section covers what that automated suite cannot: a manual baseline comparison for H1 and a targeted before/after comparison for H3.

H1. Five pairs were run through the full pipeline and three baseline conditions per the evaluation plan (§3.7): Baseline 1, the time-series signal alone; Baseline 2, the sentiment signal alone; Baseline 3, raw data fed directly to Mistral with no specialist processing.

SUMMARY TABLE					
Pair	Pipeline	Baseline1 (price)	Baseline2 (text)	Baseline3 (LLM)	
EURUSD 1h	bullish, 0.648	bullish, 0.508	neutral, 0.029	bullish, 0.700	
GBPUSD 1h	bullish, 0.629	bearish, 0.515	neutral, 0.052	bullish, 0.700	
USDJPY 1h	bullish, 0.730	bullish, 0.519	neutral, 0.062	bullish, 0.700	
AUDUSD 1d	bullish, 0.679	bullish, 0.517	bullish, 0.135	ERROR, 0.000	
USDCAD 1d	bullish, 0.846	bullish, 0.574	neutral, 0.040	ERROR, 0.000	

Table 4: Summary table output from `eval_orchestrator.py` comparing integrated pipeline against 3 baselines, full reasoning logs for Baseline 3 are in Appendix D

Baseline 1 agrees with the pipeline on 4/5 pairs, diverging on GBPUSD (pipeline bullish 0.629 medium conflict, Baseline 1 bearish 0.515), showing fusion can override a single strong modality when the others disagree.

Baseline 2 agrees on only 1/5 pairs and reports near-zero confidence (0.029 to 0.062) on four of them. This is not a defect in the comparison, it confirms the sentiment modality's own documented limitation (§4.2): weak signal deliberately collapses to neutral via a threshold band, now visibly reproduced across four live pairs.

Baseline 3 succeeded on 3/5 pairs but returned identical 0.700 confidence and medium conflict throughout. Its reasoning text is visibly templated, reusing “retail short positioning” phrasing across different pairs. Crucially, Baseline 3 failed outright on AUDUSD and USDCAD due to JSON parsing errors. Across three independent executions, the baseline failed on 4/5 pairs at least once, whereas the pipeline never failed. These parsing errors validates H1: the raw LLM cannot reliably structure outputs without the pipeline's orchestration.

DIRECTIONAL AGREEMENT WITH PIPELINE	
Baseline 1 (price-only)	4/5 pairs agree with pipeline stance
Baseline 2 (text-only)	2/5 pairs agree with pipeline stance
Baseline 3 (naive LLM)	2/5 pairs agree with pipeline stance

Figure 14: Directional agreement between the three baselines and the pipeline from `eval_orchestrator.py`

Reasoning outputs were manually scored for coherence on a 1 to 5 scale (Appendix E). Outputs were scored only where reasoning text exists:

Pair	Pipeline	Baseline 3	Notes
EURUSD 1H	5	4	Baseline cites unsupported macro claim (ECB hopes), not grounded in the supplied data. Pipeline names each modality individually.
GBPUSD 1H	5	3	Baseline reuses generic phrasing that could apply to any pair, no pair-specific grounding. Pipeline explains its own conflict source.
USDJPY 1H	4	3	Pipeline slightly less specific this run, doesn't name the exact pattern. Baseline again reuses the same generic phrasing as GBPUSD.
AUDUSD 1D	5	(error)	Pipeline explains high conflict via GRU's reduced confidence weighting well.
USDCAD 1D	5	(error)	Pipeline ties GRU's reduced influence directly to its sub-1.0 confidence.

Table 5: Coherence scoring (1-5 scale) of the pipeline versus Baseline 3. Scores are derived from the full reasoning logs in Appendix D. Table based on the rubric provided by `eval_orchestrator.py` in Appendix E

Pipeline averaged 4.8 (below the prior run's 5.0, evidence the scoring is not defaulting to maximum). Baseline 3 averaged 3.3 across successful outputs, or 2.4 if failures are scored as 1 rather than excluded. The delta holds either way.

Pipeline outputs continue to calibrate language to each signal's own confidence, for example “the GRU model confidence is below 1.0, carrying reduced influence for this pair” on USDCAD, consistent with the conflict-aware reasoning fix (§4.6) generalising beyond its original purpose.

A fuller 10-run consistency check on EURUSD 1h returned 100% consistency (all bullish), confidence mean 0.647, standard deviation 0.003, conflict low throughout. Run 1 took 64.0s against 28.9~39.8s for the remaining nine, a further confirmation of the model caching claim (§4.1, §5.3; Appendix F).

H2. The conflict-scoring logic itself is verified at the unit level in §5.2, where `compute_conflict()` is tested against each conflict band directly. The baseline comparison above provides a live illustration of the same mechanism: the two low-conflict outputs (GBPUSD, USDCAD) read as decisive, while each of the three high-conflict outputs explicitly names which modality's influence was discounted and why. USDJPY's reasoning, for instance, states that news sentiment “carries reduced influence

for this pair” rather than treating all four signals as equally weighted. This is consistent with the conflict score shaping the LLM's language, not just its stance.

H3. The weight-update rule was verified against eight configured cases spanning every documented branch: agreeing/correct (boosted), agreeing/incorrect (penalised), disagreeing/correct (unchanged), unanimous-and-wrong (no update), plus floor and ceiling enforcement; all eight passed. The unanimous-wrong case is the design decision worth highlighting: when every modality agreed on a stance that still turned out wrong, there is no differential signal to attribute the miss to, so the rule correctly withholds an update rather than penalising indiscriminately.

Event	news	chart	timeseries	positioning
Init	1.000	1.000	1.000	1.000
1	1.100▲	1.100▲	1.100▲	1.000
2	1.200▲	1.100	1.200▲	1.100▲
3	1.300▲	1.200▲	1.300▲	1.200▲
4	1.200▼	1.100▼	1.400▲	1.100▼
5	1.300▲	1.200▲	1.400	1.200▲
6	1.400▲	1.300▲	1.500▲	1.300▲
7	1.300▼	1.400▲	1.400▼	1.200▼
8	1.400▲	1.500▲	1.400	1.300▲
9	1.500▲	1.600▲	1.500▲	1.300
10	1.500	1.600	1.500	1.300
11	1.600▲	1.700▲	1.600▲	1.300
12	1.700▲	1.800▲	1.600	1.400▲
13	1.600▼	1.700▼	1.700▲	1.300▼
14	1.700▲	1.800▲	1.800▲	1.400▲
15	1.800▲	1.900▲	1.900▲	1.500▲

Modality	Initial	Final	Delta	% share
news	1.000	1.800	+0.800	25%
chart	1.000	1.900	+0.900	27%
timeseries	1.000	1.900	+0.900	27%
positioning	1.000	1.500	+0.500	21%

Updates applied : 14 / 15
 Skipped (no-op) : 1
 (skipped = uncertain outcome or unanimous-wrong events)

Table 6: Simulated 15-event feedback weight trajectory per modality and summary statistics, generated by `eval_feedback.py`

A 15-event simulated feedback trajectory shows all four modalities’ weights trending upward overall, with 14 of 15 events applying an update and the remaining event skipped as a no-op. Chart and timeseries carried the largest cumulative increase (+0.900 each, 27% share each). News close behind (+0.800, 25%) and positioning the smallest (+0.500, 21%).

The direct test of H3 compares ten runs at default weights against ten runs at post-feedback weights:

COMPARISON		
Metric	Default	Post-Feedback
Consistency rate	60%	80%
Most common stance	neutral	neutral
Confidence mean	0.514	0.506
Confidence std dev	0.020	0.014

Table 7: Stance consistency and confidence variance across ten runs at default weights versus ten runs at post-feedback weights (EURUSD 1h), output by `eval_feedback.py`

Unlike the orchestrator-level consistency check in §5.3 (Section 1), run separately and returning a 100%-bullish result, this comparison returned a neutral stance at a lower baseline consistency rate. Because every modality fetches live data at call time, RSS headlines, current OHLC, current retail positioning; two tests on the same nominal pair and timeframe run at different times are not testing identical inputs because the underlying market conditions had moved between runs. This particular run's lower default-weight consistency (60%, rather than a ceiling at 100%) leaves genuine room to observe change, both metrics move in the direction H3 predicts: consistency rate rises 20 points and confidence variance narrows ($0.020 \rightarrow 0.014$). This differs from an earlier run of the same comparison, which returned 100% consistency under both conditions due to a ceiling effect. Taken together, this is a more informative test of H3 than a ceiling-limited run allows. Though still a single 10-run comparison against simulated rather than real feedback and not conclusive on its own.

5.5. Testing on Data: GRU Configuration Sweep

The tuning sweep that produced the winning configuration per pair/timeframe (Table 2, §4.4) doubles as the project's main empirical evaluation: a controlled experiment rather than a pass/fail test, run across all seven pairs on both timeframes, 224 models trained in total.

The clearest result from this sweep is not uniform and is worth mentioning. Six of the seven daily-timeframe models achieved 80.20% to 84.51% directional accuracy, with the notable exception of USDJPY_1d dropping to 53.80%. While hourly-timeframe models clustered much lower between 52.91% and 57.82%, barely above a random baseline.

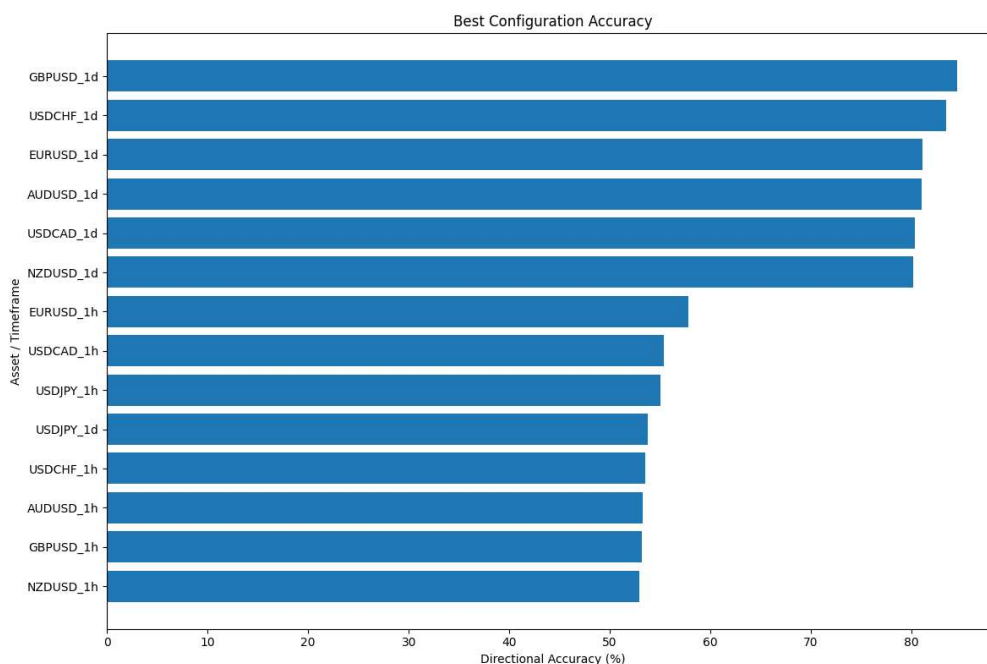


Figure 15: Validation accuracy of the selected GRU configuration for each currency pair and timeframe, derived from the 224-model GRU_tuning.py

Reading alongside the feature-set pattern already noted in §4.4 (every daily winner except GBPUSD and USDJPY uses plain OHLC, every hourly winner needs the added indicators), the results suggest that daily price action carries a stronger learnable signal, while hourly data is noisier. The indicators appear to partially compensate for the weaker hourly signal, narrowing but not eliminating the

performance gap. This is a genuine, non-obvious empirical finding rather than a restatement of the efficient-market-hypothesis caveat from the prototype stage.

One methodological limitation applies to this experiment itself and should be stated alongside it rather than left implicit: all 224 configurations were selected using a single fixed 80/20 chronological split, not walk-forward validation. This means each winning configuration's advantage is demonstrated on one contiguous recent validation window rather than averaged across several. Given the sweep already involves 224 trained models, walk-forward validation across all of them was not computationally practical.

5.6. User Acceptance Testing

A short questionnaire was used to evaluate the dashboard from a user's perspective, assessing aspects that unit and system tests cannot because they measure structural correctness rather than whether users can readily understand the output. The questionnaire comprised three 5-point Likert items evaluating the clarity of the fusion output, the adequacy of the reasoning text and the explanation of conflicting modality signals. Including one open-ended question inviting respondents to identify anything they found confusing or difficult to interpret (Appendix G.1).

Based on the full Likert results (Appendix G.2), item 1 returned a mostly-Agree response, suggesting the top-level output card is legible on its own. Items 2 and 3 were more mixed, clustering around neutral-to-agree/disagree, indicating the current output doesn't fully surface why a stance was reached or how conflicting signals were weighed against each other, despite that information existing internally (the reasoning string, the conflict score, the raw imbalance value).

The open-ended responses (Appendix G.3) converged on four points, two of which reinforce the Likert findings and two of which surface issues not captured by the scaled questions at all:

- Reasoning text felt too brief to justify the stance on its own (supports item 2)
- No visible indication of how much the modalities disagreed, beyond the conflict-level label (supports item 3)
- A run's feedback becomes permanently inaccessible once the session moves on: the history panel showed past rows with a PENDING badge but no path back into the outcome form, so closing the tab or starting a new run before submitting left that row's outcome unrecoverable
- History is hard-capped at five rows with no way to page further back

5.7. Critical Discussion

Bringing the test results, tuning results and UAT responses together against the project's stated aims:

What's working. The per-modality fallback design holds up under direct failure-injection testing and the feedback loop's weight-update logic is fully implemented and exhaustively branch-tested (§5.2). The fusion layer's consistency across repeated runs is measured, not assumed and the baseline comparison in §5.4 provides evidence supporting H1: that the orchestrated architecture

outperforms an unorchestrated language model on the same data. And the positioning modality's fixed 60% threshold and linear confidence scaling (§4.5), both explicitly flagged as naive at the design stage, remain unchanged, but are not left unmitigated: a modality that performs poorly for a given pair is down-weighted over time by the same feedback mechanism, so neither needs fixing in isolation for the system as a whole to self-correct. H2's conflict-aware reasoning is supported directly, high- and low-conflict outputs reading differently in a way that tracks the computed conflict score (§5.4). H3's evidence remains the most mixed of the three: one ten-run comparison showed the predicted direction, consistency up and variance down, while a second showed no observable change under a low-conflict, high-agreement snapshot, better read as encouraging than conclusive (§5.4).

Testing surfaced six specific issues, four through UAT and two through personal testing during development. Each was addressed directly; Table 8 lists the issue, the evidence that surfaced it and the resulting change.

Issue	Evidence	Resolution
Reasoning text too brief	UAT item 2 + open-ended	Expanded fusion prompt's reasoning instruction from 2–3 sentences to 4–5
Conflict severity not visible	UAT item 3 + open-ended	Raw imbalance value now returned alongside the conflict-level bucket; surfaced on the dashboard below the confidence bar
Feedback unreachable after session moves on	Open-ended	History rows now link to their own outcome form individually, instead of only the current run being reachable
History capped at 5	Open-ended	Pagination added to the history panel
RSS headline duplication across feeds	Personal testing notes	Exact-match deduplication added on normalised headline text; catches identical headlines
Chart save-path collision under concurrent use	Personal testing notes	Chart images now saved to a unique filename per run rather than a single fixed path

Table 8: Issues identified through testing and the resulting changes

One issue remains open: the daily/hourly accuracy split (§5.5) shows the time-series modality's reliability is timeframe-dependent in a way the dashboard still does not communicate to the user. Two items were deliberately assessed and not carried forward, with justification rather than by omission. First, chart output is not surfaced as an inline image on the dashboard by design. The system is built around structured, LLM-mediated fusion output rather than raw visual inspection; adding an inline chart image would work against that design intent. Second, the fusion prompt's current scope (asset and timeframe only, without volatility regime or recent-trend context) is a deliberate boundary rather than a technical limitation. `validate_output()` and the retry loop already enforce the output schema independently of how much context is provided, so enriching the prompt would not threaten structural consistency. This is therefore a deliberate design decision rather than a limitation imposed by the architecture.

6. Conclusion

FusionVerge set out from a specific claim: the retail trader's problem is not a lack of data but the absence of structured interpretation when signals conflict. Huang et al. (2021) gave that claim a formal basis, proving that combining modalities achieves lower population risk than any subset alone. Lee and Yoo (2019) found the same holds empirically in finance and called explicitly for more interpretable systems with broader data coverage. FusionVerge answers that call directly: four specialist models feed a locally hosted LLM that reports a stance, a confidence score, a conflict level and a per-modality breakdown, something neither FinAgent nor MountainLion give their users.

This fragmentation was never just an inconvenience. Roetzel and Fehrenbacher (2019) found that adding more inputs without structure actively worsens decision quality, and Caporale and Plastun (2023) showed that technical indicators taken in isolation are subjective enough to mislead even attentive traders. Multi-modal fusion is the answer the theory points to; FusionVerge is one attempt at building it for a user who is not a quant.

The evaluation in §5 gives each of the three hypotheses a clear verdict. H1 is supported: across repeated baseline comparisons, the pipeline consistently produced grounded, pair-specific reasoning, while a naive, unorchestrated LLM given the same underlying data repeatedly defaulted to generic phrasing and, on several occasions, failed to produce structured output at all. This is consistent with Kim et al.'s (2024) finding that LLM reasoning degrades once context is incomplete: the naive baseline receives the same data without the specialist processing that keeps that context clean. H2 is also supported, the conflict score is verified at the unit level and shown, in the same comparisons, shaping the LLM's language rather than just its stance. H3 is the weakest of the three: the weight-update rule passed every branch test, but the live consistency comparisons gave mixed results across separate runs, encouraging in one, flat in another, both resting on simulated rather than real outcomes over a small number of runs.

Testing also surfaced six concrete usability issues during development, from short reasoning text and invisible conflict severity to unreachable feedback, capped history, duplicate headlines and colliding chart files. All six were resolved into the version described in this report; none are outstanding.

Two further refinements follow from what testing already showed but were left for future work. Headline deduplication currently matches identical text only, so reworded coverage of the same story across feeds can still inflate the sentiment modality's confidence; similarity-based matching would close that gap, though it is minor next to the items below. More substantially, the GRU configuration sweep selected all 224 configurations on a single fixed chronological split, and daily-timeframe models substantially outperformed hourly ones, some of which sat barely above chance. A targeted walk-forward re-validation of just the selected daily and hourly winners, rather than repeating the full sweep, would confirm whether that gap reflects a genuine difference in how learnable daily versus hourly price action is, or is partly an artefact of validating on a single window.

Four further extensions are architectural rather than incremental, and none carry evaluation evidence attached, since the evaluation in this report was scoped to the nearer-term items above. Positioning currently relies on retail long/short ratios alone; institutional data such as Commitment of Traders reports would add a second, independent view of crowd positioning. The four modalities

run sequentially despite not depending on each other's output, so parallel execution is a straightforward way to reduce analysis time. The fusion layer runs a general-purpose local model; a finance-tuned LLM in place of Mistral 7B could plausibly produce sharper, less templated reasoning, following the same domain-adaptation logic that made FinBERT preferable to a generic sentiment model. Last, the system was evaluated on forex pairs throughout, though the original design targets both forex and equity traders, so generalising to equities and other asset classes remains the clearest way to reach that original scope.

A deployment note is worth adding here too. Of the four models in the pipeline, only the fusion layer calls a general-purpose LLM, and it does so once per analysis; replacing the locally hosted, quantised Mistral 7B with a hosted LLM API would meaningfully reduce local compute requirements without touching any other component. FinBERT, YOLOv8 and the GRU models are not general-purpose LLMs and would still need to run locally or on a hosted VPS either way, so this swap addresses the heaviest single dependency rather than the system's full compute footprint.

None of this changes what the system already demonstrates. Fazlollahi et al. (1997) frame adaptive decision support as a component that monitors and adjusts around the decision maker rather than replacing them, the same principle FusionVerge's feedback loop follows without invoking reinforcement learning. Lee and Yoo (2019) called for a system that combines varied financial data and explains itself rather than optimising for accuracy alone; the literature review found no accessible, user-facing tool doing that as of this project's writing. The retail trader's problem was never a lack of data. It was the absence of structure when signals disagree. The conflict score and fusion reasoning built to address that directly are the parts of the system that held up best across testing. The extensions above build on that foundation; they do not patch a gap in it.

7. Appendices

Appendix A. Prototype Model Comparison

A.1 Sentiment Model comparison

	Headline	ProsusAI Label	ProsusAI Conf	FinBERT-Tone Label	FinBERT-Tone Conf	Agreement
0	Gold holds losses as Iran impasse keeps rate hike bets high...	negative	0.6100	Negative	0.9998	T
1	Trump says he'll let Warsh 'do what he wants to do' with int...	neutral	0.6295	Neutral	0.9595	T
2	China's Xi says the basis of trust between Beijing and Mosco...	neutral	0.5389	Positive	1.0000	F
3	EU reaches provisional trade deal with US, seeks to head off...	positive	0.9201	Neutral	0.9994	F
4	US 30-Year Yield Hits Highest Since 2007 on Inflation Angst...	positive	0.8012	Positive	1.0000	T
5	Bitcoin surges past \$70,000 on renewed institutional buying ...	positive	0.9397	Positive	1.0000	T
6	Asian markets mixed ahead of key US inflation data release...	negative	0.8807	Negative	0.7287	T

Figure A.1: Prototype comparison of ProsusAI/FinBERT and FinBERT-Tone on identical financial headlines, illustrating similar sentiment classifications but different confidence distributions that informed the final sentiment model selection.

A.2. Chart/Image Model comparison

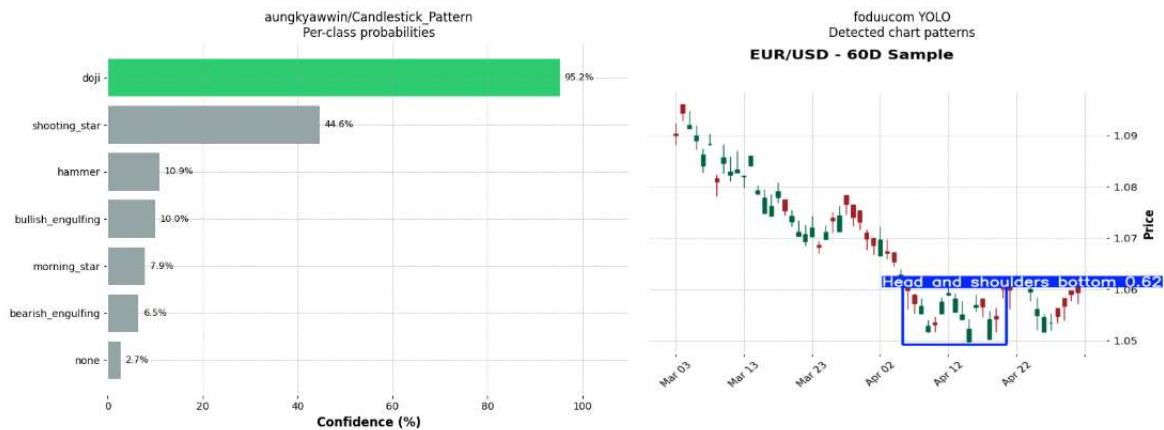


Figure A.2: Prototype comparison between a candlestick classification model and a YOLOv8-based chart pattern detector, demonstrating the difference between single-candle recognition and higher-level structural pattern detection used for the implemented pipeline.

A.3. Time-series Model comparison

Evaluation on Validation Set:

Custom LSTM	Directional Accuracy: 55.95%	Signal: -1	Conf: 0.438
Custom GRU	Directional Accuracy: 55.95%	Signal: -1	Conf: 0.479
Rogendo Pretrained LSTM	Directional Accuracy: 55.96%	Signal: -1	Conf: 0.459

Figure A.3: Prototype comparison of a pretrained LSTM (Rogendo/forex-lstm-models), a custom LSTM and custom GRU, showing comparable validation performance that informed the final time-series model selection.

A.4. Fusion Model comparison

Fusion Model Comparison:		
Criterion	Mistral	Llama3
Stance	bullish	bullish
Confidence	0.648	0.76
Conflict level	medium	medium
Stance agreement?	YES	

Mistral Reasoning:

The overall stance is bullish, despite some conflicting signals. The bullish news sentiment and chart pattern are supported by the contrarian retail positioning. However, it's important to note that the time-series analysis shows a bearish signal with lower confidence.

Llama3 Reasoning:

Despite some conflicting signals from the time-series model, the overall sentiment is bullish due to positive news and a head and shoulders top pattern. However, the positioning data suggests that retail traders may be overly bearish, which could create an opportunity for contrarian trading.

Figure A.4: Prototype comparison of Mistral 7B and Llama 3 8B using identical modality inputs. Both produced the same stance, while Mistral generated more focused conflict-aware reasoning and was therefore chosen.

Appendix B. Unit Test Results

B.1. sentiment.py

```
tests/test_sentiment.py::TestFetchHeadlines::test_returns_list_for_active_pair PASSED [ 2%]
tests/test_sentiment.py::TestFetchHeadlines::test_headlines_are_strings PASSED [ 4%]
tests/test_sentiment.py::TestFetchHeadlines::test_filtered_headlines_contain_keyword PASSED [ 7%]
tests/test_sentiment.py::TestFetchHeadlines::test_unknown_pair_returns_empty PASSED [ 9%]
tests/test_sentiment.py::TestFetchHeadlines::test_max_headlines_caps_result PASSED [ 11%]
tests/test_sentiment.py::TestFetchHeadlines::test_max_headlines_zero_returns_empty PASSED [ 14%]
tests/test_sentiment.py::TestClassifyHeadlines::test_output_structure PASSED [ 16%]
tests/test_sentiment.py::TestClassifyHeadlines::test_label_is_valid PASSED [ 19%]
tests/test_sentiment.py::TestClassifyHeadlines::test_confidence_in_range PASSED [ 21%]
tests/test_sentiment.py::TestClassifyHeadlines::test_strongly_positive_headline PASSED [ 23%]
tests/test_sentiment.py::TestClassifyHeadlines::test_strongly_negative_headline PASSED [ 26%]
tests/test_sentiment.py::TestClassifyHeadlines::test_empty_input_returns_empty PASSED [ 28%]
tests/test_sentiment.py::TestClassifyHeadlines::test_one_result_per_headline PASSED [ 30%]
tests/test_sentiment.py::TestDetectCurrencyFocus::test_eur_headline_detected_as_eur PASSED [ 33%]
tests/test_sentiment.py::TestDetectCurrencyFocus::test_usd_headline_detected_as_usd PASSED [ 35%]
tests/test_sentiment.py::TestDetectCurrencyFocus::test_both_currencies_resolves_neutral PASSED [ 38%]
tests/test_sentiment.py::TestDetectCurrencyFocus::test_unrelated_headline_is_neutral PASSED [ 40%]
tests/test_sentiment.py::TestDetectCurrencyFocus::test_gbp_headline_for_gbpusd PASSED [ 42%]
tests/test_sentiment.py::TestDetectCurrencyFocus::test_jpy_headline_for_usdjpy PASSED [ 45%]
tests/test_sentiment.py::TestDetectCurrencyFocus::test_cad_headline_for_usdcad PASSED [ 47%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_positive_usd_is_bearish_eurusd PASSED [ 50%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_positive_eur_is_bullish_eurusd PASSED [ 52%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_negative_eur_is_bearish_eurusd PASSED [ 54%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_positive_gbp_is_bullish_gbpusd PASSED [ 57%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_positive_jpy_is_bearish_usdjpy PASSED [ 59%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_positive_usd_is_bullish_usdjpy PASSED [ 61%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_neutral_label_gives_zero_signal PASSED [ 64%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_unrelated_headline_gives_zero_signal PASSED [ 66%]
tests/test_sentiment.py::TestApplyCurrencyAdjustment::test_signal_key_present_on_all_items PASSED [ 69%]
tests/test_sentiment.py::TestAggregateSignal::test_empty_input_returns_neutral PASSED [ 71%]
tests/test_sentiment.py::TestAggregateSignal::test_bullish_majority_returns_positive_signal PASSED [ 73%]
tests/test_sentiment.py::TestAggregateSignal::test_bearish_majority_returns_negative_signal PASSED [ 76%]
tests/test_sentiment.py::TestAggregateSignal::test_balanced_signals_return_neutral PASSED [ 78%]
tests/test_sentiment.py::TestAggregateSignal::test_distribution_counts_labels PASSED [ 80%]
tests/test_sentiment.py::TestAggregateSignal::test_confidence_is_float_in_range PASSED [ 83%]
tests/test_sentiment.py::TestAggregateSignal::test_output_has_required_keys PASSED [ 85%]
tests/test_sentiment.py::TestGetSentimentSignal::test_returns_required_keys PASSED [ 88%]
tests/test_sentiment.py::TestGetSentimentSignal::test_signal_is_valid PASSED [ 90%]
tests/test_sentiment.py::TestGetSentimentSignal::test_confidence_in_range PASSED [ 92%]
tests/test_sentiment.py::TestGetSentimentSignal::test_distribution_has_all_labels PASSED [ 95%]
tests/test_sentiment.py::TestGetSentimentSignal::test_unknown_pair_returns_neutral PASSED [ 97%]
tests/test_sentiment.py::TestGetSentimentSignal::test_max_headlines_respected PASSED [100%]

===== 42 passed in 15.70s =====
```

Figure B.1: Full pytest output for sentiment.py

B.2. chart.py

```
tests/test_chart.py::TestFetchOhlcv::test_returns_dataframe PASSED [ 4%]
tests/test_chart.py::TestFetchOhlcv::test_has_ohlcv_columns PASSED [ 9%]
tests/test_chart.py::TestFetchOhlcv::test_no_volume_column PASSED [ 14%]
tests/test_chart.py::TestFetchOhlcv::test_no_nulls PASSED [ 19%]
tests/test_chart.py::TestFetchOhlcv::test_non_empty PASSED [ 23%]
tests/test_chart.py::TestFetchOhlcv::test_index_is_datetime PASSED [ 28%]
tests/test_chart.py::TestFetchOhlcv::test_unsupported_pair_raises PASSED [ 33%]
tests/test_chart.py::TestGenerateChart::test_creates_image_file PASSED [ 38%]
tests/test_chart.py::TestGenerateChart::test_returns_savepath PASSED [ 42%]
tests/test_chart.py::TestGenerateChart::test_image_has_nonzero_size PASSED [ 47%]
tests/test_chart.py::TestGenerateChart::test_missing_column_raises PASSED [ 52%]
tests/test_chart.py::TestRunYoloInference::test_returns_required_keys PASSED [ 57%]
tests/test_chart.py::TestRunYoloInference::test_signal_is_valid PASSED [ 61%]
tests/test_chart.py::TestRunYoloInference::test_confidence_in_range PASSED [ 66%]
tests/test_chart.py::TestRunYoloInference::test_pattern_is_known_label PASSED [ 71%]
tests/test_chart.py::TestRunYoloInference::test_no_detection_returns_neutral PASSED [ 76%]
tests/test_chart.py::TestRunYoloInference::test_signal_matches_pattern PASSED [ 80%]
tests/test_chart.py::TestGetChartSignal::test_returns_required_keys PASSED [ 85%]
tests/test_chart.py::TestGetChartSignal::test_signal_is_valid PASSED [ 90%]
tests/test_chart.py::TestGetChartSignal::test_confidence_in_range PASSED [ 95%]
tests/test_chart.py::TestGetChartSignal::test_unsupported_pair_raises PASSED [100%]

===== 21 passed in 12.43s =====
```

Figure B.2: Full pytest output for chart.py

B.3. timeseries.py

```
tests/test_timeseries.py::TestFetchOhlcv::test_returns_dataframe PASSED [ 2%]
tests/test_timeseries.py::TestFetchOhlcv::test_has_ohlcv_columns PASSED [ 5%]
tests/test_timeseries.py::TestFetchOhlcv::test_no_volume_column PASSED [ 8%]
tests/test_timeseries.py::TestFetchOhlcv::test_no_nulls PASSED [ 11%]
tests/test_timeseries.py::TestFetchOhlcv::test_non_empty PASSED [ 14%]
tests/test_timeseries.py::TestFetchOhlcv::test_unsupported_pair_raises PASSED [ 17%]
tests/test_timeseries.py::TestFetchOhlcv::test_unsupported_timeframe_raises PASSED [ 20%]
tests/test_timeseries.py::TestComputeIndicators::test_adds_indicator_columns PASSED [ 23%]
tests/test_timeseries.py::TestComputeIndicators::test_no_nulls_after_compute PASSED [ 26%]
tests/test_timeseries.py::TestComputeIndicators::test_row_count_reduced_by_warmup PASSED [ 29%]
tests/test_timeseries.py::TestComputeIndicators::test_rsi_in_valid_range PASSED [ 32%]
tests/test_timeseries.py::TestComputeIndicators::test_original_ohlcv_columns_preserved PASSED [ 35%]
tests/test_timeseries.py::TestComputeIndicators::test_does_not_modify_input PASSED [ 38%]
tests/test_timeseries.py::TestLoadBestConfig::test_returns_dict PASSED [ 41%]
tests/test_timeseries.py::TestLoadBestConfig::test_has_required_keys PASSED [ 44%]
tests/test_timeseries.py::TestLoadBestConfig::test_features_is_list PASSED [ 47%]
tests/test_timeseries.py::TestLoadBestConfig::test_lookback_is_positive_int PASSED [ 50%]
tests/test_timeseries.py::TestLoadBestConfig::test_features_are_valid_columns PASSED [ 52%]
tests/test_timeseries.py::TestLoadBestConfig::test_missing_json_raises PASSED [ 55%]
tests/test_timeseries.py::TestLoadBestConfig::test_unknown_pair_raises PASSED [ 58%]
tests/test_timeseries.py::TestLoadGru::test_returns_model_and_scaler PASSED [ 61%]
tests/test_timeseries.py::TestLoadGru::test_missing_model_raises PASSED [ 64%]
tests/test_timeseries.py::TestLoadGru::test_missing_scaler_raises PASSED [ 67%]
tests/test_timeseries.py::TestRunGruInference::test_input_shape_is_correct PASSED [ 70%]
tests/test_timeseries.py::TestRunGruInference::test_returns_required_keys PASSED [ 73%]
tests/test_timeseries.py::TestRunGruInference::test_signal_is_valid PASSED [ 76%]
tests/test_timeseries.py::TestRunGruInference::test_direction_is_0_or_1 PASSED [ 79%]
tests/test_timeseries.py::TestRunGruInference::test_confidence_in_range PASSED [ 82%]
tests/test_timeseries.py::TestRunGruInference::test_confidence_consistent_with_direction_probability PASSED [ 85%]
tests/test_timeseries.py::TestRunGruInference::test_insufficient_rows_raises PASSED [ 88%]
tests/test_timeseries.py::TestGetTimeseriesSignal::test_returns_required_keys PASSED [ 91%]
tests/test_timeseries.py::TestGetTimeseriesSignal::test_signal_is_valid PASSED [ 94%]
tests/test_timeseries.py::TestGetTimeseriesSignal::test_confidence_in_range PASSED [ 97%]
tests/test_timeseries.py::TestGetTimeseriesSignal::test_unsupported_pair_raises PASSED [100%]

===== 34 passed in 9.47s =====
```

Figure B.3: Full pytest output for timeseries.py

B.4. positioning.py

```
tests/test_positioning.py::TestApplyContrarianRule::test_65_short_produces_bullish_signal PASSED [ 4%]
tests/test_positioning.py::TestApplyContrarianRule::test_65_long_produces_bearish_signal PASSED [ 8%]
tests/test_positioning.py::TestApplyContrarianRule::test_at_threshold_short_is_bullish PASSED [ 12%]
tests/test_positioning.py::TestApplyContrarianRule::test_below_threshold_is_neutral PASSED [ 16%]
tests/test_positioning.py::TestApplyContrarianRule::test_balanced_50_50_is_neutral PASSED [ 20%]
tests/test_positioning.py::TestApplyContrarianRule::test_confidence_scales_with_percentage PASSED [ 24%]
tests/test_positioning.py::TestApplyContrarianRule::test_confidence_in_range PASSED [ 28%]
tests/test_positioning.py::TestApplyContrarianRule::test_returns_required_keys PASSED [ 32%]
tests/test_positioning.py::TestApplyContrarianRule::test_note_is_string PASSED [ 36%]
tests/test_positioning.py::TestApplyContrarianRule::test_missing_keys_use_defaults PASSED [ 40%]
tests/test_positioning.py::TestExtractPairData::test_extracts_correct_pair PASSED [ 44%]
tests/test_positioning.py::TestExtractPairData::test_case_insensitive_match PASSED [ 48%]
tests/test_positioning.py::TestExtractPairData::test_missing_pair_returns_none PASSED [ 52%]
tests/test_positioning.py::TestExtractPairData::test_empty_symbols_returns_none PASSED [ 56%]
tests/test_positioning.py::TestLoginLogout::test_login_returns_session_string PASSED [ 60%]
tests/test_positioning.py::TestLoginLogout::test_logout_does_not_raise PASSED [ 64%]
tests/test_positioning.py::TestFetchCommunityOutlook::test_returns_list PASSED [ 68%]
tests/test_positioning.py::TestFetchCommunityOutlook::test_non_empty PASSED [ 72%]
tests/test_positioning.py::TestFetchCommunityOutlook::test_each_symbol_has_percentages PASSED [ 76%]
tests/test_positioning.py::TestFetchCommunityOutlook::test_percentages_sum_to_100 PASSED [ 80%]
tests/test_positioning.py::TestGetPositioningSignal::test_returns_required_keys PASSED [ 84%]
tests/test_positioning.py::TestGetPositioningSignal::test_signal_is_valid PASSED [ 88%]
tests/test_positioning.py::TestGetPositioningSignal::test_confidence_in_range PASSED [ 92%]
tests/test_positioning.py::TestGetPositioningSignal::test_contrarian_signal_correct_for_three_pairs PASSED [ 96%]
tests/test_positioning.py::TestGetPositioningSignal::test_fallback_on_bad_credentials PASSED [100%]

===== 25 passed in 12.45s =====
```

Figure B.4: Full pytest output for positioning.py

B.5. feedback.py

```
tests/test_feedback.py::TestLoadWeights::test_returns_default_when_no_history PASSED [ 2%]
tests/test_feedback.py::TestLoadWeights::test_returns_stored_weights_when_history_exists PASSED [ 5%]
tests/test_feedback.py::TestLoadWeights::test_missing_modality_key_backfilled_with_default PASSED [ 7%]
tests/test_feedback.py::TestLoadWeights::test_all_modality_keys_always_present PASSED [ 10%]
tests/test_feedback.py::TestLoadWeights::test_queries_correct_pair PASSED [ 12%]
tests/test_feedback.py::TestApplyWeights::test_returns_new_dict_does_not_mutate PASSED [ 15%]
tests/test_feedback.py::TestApplyWeights::test_effective_confidence_equals_raw_times_weight PASSED [ 17%]
tests/test_feedback.py::TestApplyWeights::test_effective_confidence_clamped_to_one PASSED [ 20%]
tests/test_feedback.py::TestApplyWeights::test_raw_confidence_preserved PASSED [ 22%]
tests/test_feedback.py::TestApplyWeights::test_weight_key_added_to_output PASSED [ 25%]
tests/test_feedback.py::TestApplyWeights::test_default_weight_is_transparent PASSED [ 27%]
tests/test_feedback.py::TestApplyWeights::test_all_modalities_processed PASSED [ 30%]
tests/test_feedback.py::TestApplyWeights::test_signal_unchanged_by_weighting PASSED [ 32%]
tests/test_feedback.py::TestApplyWeights::test_missing_weight_uses_default PASSED [ 35%]
tests/test_feedback.py::TestComputeWeightUpdate::test_uncertain_outcome_returns_none PASSED [ 37%]
tests/test_feedback.py::TestComputeWeightUpdate::test_unanimous_wrong_returns_none PASSED [ 40%]
tests/test_feedback.py::TestComputeWeightUpdate::test_all_neutral_signals_preserves_default_weights PASSED [ 42%]
tests/test_feedback.py::TestComputeWeightUpdate::test_correct_outcome_agreeing_modality_boosted PASSED [ 45%]
tests/test_feedback.py::TestComputeWeightUpdate::test_correct_outcome_dissenter_unchanged PASSED [ 47%]
tests/test_feedback.py::TestComputeWeightUpdate::test_incorrect_outcome_agreeing_modality_penalised PASSED [ 50%]
tests/test_feedback.py::TestComputeWeightUpdate::test_floor_enforced_on_penalty PASSED [ 52%]
tests/test_feedback.py::TestComputeWeightUpdate::test_ceiling_enforced_on_boost PASSED [ 55%]
tests/test_feedback.py::TestComputeWeightUpdate::test_neutral_signal_modality_unchanged PASSED [ 57%]
tests/test_feedback.py::TestComputeWeightUpdate::test_returns_dict_with_all_modality_keys PASSED [ 60%]
tests/test_feedback.py::TestComputeWeightUpdate::test_does_not_mutate_input_weights PASSED [ 62%]
tests/test_feedback.py::TestComputeWeightUpdate::test_learning_rate_applied_correctly PASSED [ 65%]
tests/test_feedback.py::TestSaveWeights::test_inserts_row_with_correct_feedback_id PASSED [ 67%]
tests/test_feedback.py::TestSaveWeights::test_weights_serialised_as_json PASSED [ 70%]
tests/test_feedback.py::TestSaveWeights::test_execute_called_once PASSED [ 72%]
tests/test_feedback.py::TestLoadWeightHistory::test_returns_list PASSED [ 75%]
tests/test_feedback.py::TestLoadWeightHistory::test_returns_empty_list_when_no_history PASSED [ 77%]
tests/test_feedback.py::TestLoadWeightHistory::test_each_entry_has_expected_keys PASSED [ 80%]
tests/test_feedback.py::TestLoadWeightHistory::test_limit_passed_to_query PASSED [ 82%]
tests/test_feedback.py::TestLoadWeightHistory::test_queries_correct_pair PASSED [ 85%]
tests/test_feedback.py::TestWeightsAsPct::test_equal_weights_give_25_percent_each PASSED [ 87%]
tests/test_feedback.py::TestWeightsAsPct::test_all_keys_present PASSED [ 90%]
tests/test_feedback.py::TestWeightsAsPct::test_percentages_sum_to_100 PASSED [ 92%]
tests/test_feedback.py::TestWeightsAsPct::test_higher_weight_gives_higher_percentage PASSED [ 95%]
tests/test_feedback.py::TestWeightsAsPct::test_zero_total_returns_equal_split PASSED [ 97%]
tests/test_feedback.py::TestWeightsAsPct::test_returns_integer_values PASSED [100%]

===== 40 passed in 0.13s =====
```

Figure B.5: Full pytest output for feedback.py

B.6. fusion.py

```
tests/test_fusion.py::TestComputeConflict::test_all_bullish_is_low_conflict PASSED [ 1%]
tests/test_fusion.py::TestComputeConflict::test_all_bearish_is_low_conflict PASSED [ 3%]
tests/test_fusion.py::TestComputeConflict::test_two_v_two_is_high_conflict PASSED [ 5%]
tests/test_fusion.py::TestComputeConflict::test_all_neutral_is_low_conflict PASSED [ 7%]
tests/test_fusion.py::TestComputeConflict::test_neutral_excluded_from_imbalance PASSED [ 9%]
tests/test_fusion.py::TestComputeConflict::test_confidence_weighting_changes_result PASSED [ 11%]
tests/test_fusion.py::TestComputeConflict::test_medium_conflict_range PASSED [ 13%]
tests/test_fusion.py::TestComputeConflict::test_returns_tuple PASSED [ 15%]
tests/test_fusion.py::TestComputeConflict::test_conflict_level_is_valid_string PASSED [ 17%]
tests/test_fusion.py::TestComputeConflict::test_missing_modality_raises PASSED [ 19%]
tests/test_fusion.py::TestBuildFusionPrompt::test_contains_pair PASSED [ 21%]
tests/test_fusion.py::TestBuildFusionPrompt::test_contains_timeframe PASSED [ 23%]
tests/test_fusion.py::TestBuildFusionPrompt::test_contains_all_modality_signals PASSED [ 25%]
tests/test_fusion.py::TestBuildFusionPrompt::test_contains_conflict_level PASSED [ 26%]
tests/test_fusion.py::TestBuildFusionPrompt::test_contains_required_output_format PASSED [ 28%]
tests/test_fusion.py::TestBuildFusionPrompt::test_within_context_window PASSED [ 30%]
tests/test_fusion.py::TestBuildFusionPrompt::test_returns_string PASSED [ 32%]
tests/test_fusion.py::TestBuildFusionPrompt::test_different_pairs_produce_different_prompts PASSED [ 34%]
tests/test_fusion.py::TestExtractJson::test_clean_json_parsed PASSED [ 36%]
tests/test_fusion.py::TestExtractJson::test_strips_markdown_fences PASSED [ 38%]
tests/test_fusion.py::TestExtractJson::test_strips_plain_code_fence PASSED [ 40%]
tests/test_fusion.py::TestExtractJson::test_json_with_leading_text PASSED [ 42%]
tests/test_fusion.py::TestExtractJson::test_auto_closes_missing_brace PASSED [ 44%]
tests/test_fusion.py::TestExtractJson::test_no_json_raises_value_error PASSED [ 46%]
tests/test_fusion.py::TestExtractJson::test_empty_string_raises PASSED [ 48%]
tests/test_fusion.py::TestValidateOutput::test_valid_output_returns_no_issues PASSED [ 50%]
tests/test_fusion.py::TestValidateOutput::test_missing_stance_flagged PASSED [ 51%]
tests/test_fusion.py::TestValidateOutput::test_missing_confidence_flagged PASSED [ 53%]
tests/test_fusion.py::TestValidateOutput::test_missing_conflict_level_flagged PASSED [ 55%]
tests/test_fusion.py::TestValidateOutput::test_missing_reasoning_flagged PASSED [ 57%]
tests/test_fusion.py::TestValidateOutput::test_invalid_stance_value_flagged PASSED [ 59%]
tests/test_fusion.py::TestValidateOutput::test_invalid_conflict_level_flagged PASSED [ 61%]
tests/test_fusion.py::TestValidateOutput::test_confidence_as_string_flagged PASSED [ 63%]
tests/test_fusion.py::TestValidateOutput::test_confidence_above_one_flagged PASSED [ 65%]
tests/test_fusion.py::TestValidateOutput::test_confidence_below_zero_flagged PASSED [ 67%]
tests/test_fusion.py::TestValidateOutput::test_confidence_exactly_zero_valid PASSED [ 69%]
tests/test_fusion.py::TestValidateOutput::test_confidence_exactly_one_valid PASSED [ 71%]
tests/test_fusion.py::TestValidateOutput::test_missing_signal_subkey_flagged PASSED [ 73%]
tests/test_fusion.py::TestValidateOutput::test_all_valid_stances_accepted PASSED [ 75%]
tests/test_fusion.py::TestValidateOutput::test_all_valid_conflict_levels_accepted PASSED [ 76%]
tests/test_fusion.py::TestCallLlmRetry::test_valid_response_returned_on_first_attempt PASSED [ 78%]
tests/test_fusion.py::TestCallLlmRetry::test_retries_on_malformed_json PASSED [ 80%]
tests/test_fusion.py::TestCallLlmRetry::test_retries_on_validation_failure PASSED [ 82%]
tests/test_fusion.py::TestCallLlmRetry::test_returns_default_after_all_retries_exhausted PASSED [ 84%]
tests/test_fusion.py::TestCallLlmRetry::test_returns_default_on_ollama_exception PASSED [ 86%]
tests/test_fusion.py::TestCallLlmRetry::test_default_output_has_required_keys PASSED [ 88%]
tests/test_fusion.py::TestGetFusionOutput::test_returns_required_keys PASSED [ 90%]
tests/test_fusion.py::TestGetFusionOutput::test_missing_modality_raises PASSED [ 92%]
tests/test_fusion.py::TestGetFusionOutput::test_stance_is_valid PASSED [ 94%]
tests/test_fusion.py::TestGetFusionOutput::test_confidence_in_range PASSED [ 96%]
tests/test_fusion.py::TestGetFusionOutput::test_all_neutral_modalities_completes_without_error PASSED [ 98%]
tests/test_fusion.py::TestGetFusionOutput::test_live_llm_call PASSED [100%]

===== 52 passed in 60.57s (0:01:00) =====
```

Figure B.6: Full pytest output for fusion.py

B.7. orchestrator.py

```
tests/test_orchestrator.py::TestOrchestratorInit::test_init_loads_sentiment_model PASSED [ 3%]
tests/test_orchestrator.py::TestOrchestratorInit::test_init_loads_yolo_model PASSED [ 6%]
tests/test_orchestrator.py::TestOrchestratorInit::test_models_stored_as_instance_variables PASSED [ 9%]
tests/test_orchestrator.py::TestOrchestratorInit::test_db_conn_defaults_to_none PASSED [ 12%]
tests/test_orchestrator.py::TestOrchestratorInit::test_db_conn_stored_when_provided PASSED [ 16%]
tests/test_orchestrator.py::TestOrchestratorCallsModalities::test_calls_sentiment PASSED [ 19%]
tests/test_orchestrator.py::TestOrchestratorCallsModalities::test_calls_chart PASSED [ 22%]
tests/test_orchestrator.py::TestOrchestratorCallsModalities::test_calls_timeseries PASSED [ 25%]
tests/test_orchestrator.py::TestOrchestratorCallsModalities::test_calls_positioning PASSED [ 29%]
tests/test_orchestrator.py::TestOrchestratorCallsModalities::test_calls_fusion PASSED [ 32%]
tests/test_orchestrator.py::TestOrchestratorCallsModalities::test_sentiment_called_with_preloaded_pipe PASSED [ 35%]
tests/test_orchestrator.py::TestOrchestratorCallsModalities::test_chart_called_with_preloaded_yolo PASSED [ 38%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_returns_dict PASSED [ 41%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_contains_fusion_keys PASSED [ 45%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_contains_metadata_keys PASSED [ 48%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_contains_weights_key PASSED [ 51%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_metadata_values_correct PASSED [ 54%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_contains_modality_outputs PASSED [ 58%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_modality_outputs_have_signal_and_confidence PASSED [ 61%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_stance_is_valid PASSED [ 64%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_confidence_in_range PASSED [ 67%]
tests/test_orchestrator.py::TestOrchestratorOutput::test_date_is_string PASSED [ 70%]
tests/test_orchestrator.py::TestOrchestratorValidation::test_unsupported_pair_raises PASSED [ 74%]
tests/test_orchestrator.py::TestOrchestratorValidation::test_unsupported_timeframe_raises PASSED [ 77%]
tests/test_orchestrator.py::TestOrchestratorValidation::test_modality_failure_uses_neutral_fallback PASSED [ 80%]
tests/test_orchestrator.py::TestOrchestratorValidation::test_all_modalities_failing_still_returns_result PASSED [ 83%]
tests/test_orchestrator.py::TestOrchestratorIntegration::test_eurusd_returns_valid_output PASSED [ 87%]
tests/test_orchestrator.py::TestOrchestratorIntegration::test_gbpusd_returns_valid_output PASSED [ 90%]
tests/test_orchestrator.py::TestOrchestratorIntegration::test_model_reuse_across_calls PASSED [ 93%]
tests/test_orchestrator.py::TestOrchestratorIntegration::test_consistency_five_consecutive_runs PASSED [ 96%]
tests/test_orchestrator.py::TestOrchestratorIntegration::test_result_contains_weights_from_db_or_default PASSED [100%]

===== 31 passed in 416.40s (0:06:56) =====
```

Figure B.7: Full pytest output for orchestrator.py

B.8. database.py

```
tests/test_database.py::TestInitDb::test_creates_feedback_log_table PASSED [ 4%]
tests/test_database.py::TestInitDb::test_creates_weight_log_table PASSED [ 9%]
tests/test_database.py::TestInitDb::test_idempotent_on_repeated_calls PASSED [ 14%]
tests/test_database.py::TestInitDb::test_feedback_log_has_required_columns PASSED [ 19%]
tests/test_database.py::TestInitDb::test_weight_log_has_required_columns PASSED [ 23%]
tests/test_database.py::TestInsertAnalysis::test_returns_integer_id PASSED [ 28%]
tests/test_database.py::TestInsertAnalysis::test_stored_values_match_input PASSED [ 33%]
tests/test_database.py::TestInsertAnalysis::test_signals_stored_as_jsonb PASSED [ 38%]
tests/test_database.py::TestInsertAnalysis::test_modality_outputs_stored_as_jsonb PASSED [ 42%]
tests/test_database.py::TestInsertAnalysis::test_outcome_is_null_on_insert PASSED [ 47%]
tests/test_database.py::TestInsertAnalysis::test_multiple_inserts_get_unique_ids PASSED [ 52%]
tests/test_database.py::TestUpdateOutcome::test_outcome_written_correctly PASSED [ 57%]
tests/test_database.py::TestUpdateOutcome::test_all_valid_outcome_values_accepted PASSED [ 61%]
tests/test_database.py::TestUpdateOutcome::test_outcome_can_be_updated_twice PASSED [ 66%]
tests/test_database.py::TestFetchHistory::test_returns_only_requested_pair PASSED [ 71%]
tests/test_database.py::TestFetchHistory::test_limit_respected PASSED [ 76%]
tests/test_database.py::TestFetchHistory::test_ordered_newest_first PASSED [ 80%]
tests/test_database.py::TestFetchHistory::test_returns_empty_list_for_unknown_pair PASSED [ 85%]
tests/test_database.py::TestFetchHistory::test_required_keys_present_in_each_row PASSED [ 90%]
tests/test_database.py::TestFetchHistory::test_outcome_field_is_none_before_feedback PASSED [ 95%]
tests/test_database.py::TestFetchHistory::test_outcome_visible_in_history_after_update PASSED [100%]

===== 21 passed in 0.89s =====
```

Figure B.8: Full pytest output for database.py

Appendix C. Representative Branch-Coverage Tests

```
class TestComputeWeightUpdate:
    # No update cases / neutral
    > def test_uncertain_outcome_returns_none(self, default_weights, all_bullish_outputs):...

    def test_unanimous_wrong_returns_none(self, default_weights, all_bullish_outputs):
        """All modalities agreed bullish, outcome incorrect - no update."""
        result = compute_weight_update(default_weights, all_bullish_outputs, "incorrect", "bullish")
        assert result is None

    > def test_all_neutral_signals_preserves_default_weights(self, default_weights):...

    # Correct outcome result
    def test_correct_outcome_agreeing_modality_boosted(self, default_weights):
        """All bullish + correct outcome - all four should be boosted."""
        outputs = {m: {"signal": 1, "confidence": 0.7} for m in MODALITIES}
        # Need at least one dissenter so its not the 'unanimous' case, but for correct outcome unanimous is fine (so all boosted)
        result = compute_weight_update(default_weights, outputs, "correct", "bullish")
        assert result is not None
        for mod in MODALITIES:
            assert result[mod] == round(DEFAULT_WEIGHT + LEARNING_RATE, 3)

    > def test_correct_outcome_dissenter_unchanged(self, default_weights, mixed_outputs):...

    # Incorrect outcome
    > def test_incorrect_outcome_agreeing_modality_penalised(self, default_weights):...

    def test_floor_enforced_on_penalty(self):
        """Weights near floor must not drop below WEIGHT_FLOOR."""
        near_floor = {m: WEIGHT_FLOOR + 0.05 for m in MODALITIES}
    > outputs = {...
        result = compute_weight_update(near_floor, outputs, "incorrect", "bullish")
        assert result is not None
        for mod in ["news", "chart", "timeseries"]:
            assert result[mod] >= WEIGHT_FLOOR

    def test_ceiling_enforced_on_boost(self):
        """Weights near ceiling must not exceed WEIGHT_CEILING."""
        near_ceil = {m: WEIGHT_CEILING - 0.05 for m in MODALITIES}
        outputs = {m: {"signal": 1, "confidence": 0.7} for m in MODALITIES}
        result = compute_weight_update(near_ceil, outputs, "correct", "bullish")
        assert result is not None
        for mod in MODALITIES:
            assert result[mod] <= WEIGHT_CEILING

    > def test_neutral_signal_modality_unchanged(self, default_weights):...

    > def test_returns_dict_with_all_modality_keys(self, default_weights):...

    > def test_does_not_mutate_input_weights(self, default_weights):...

    > def test_learning_rate_applied_correctly(self, default_weights):...
```

Figure C.1: Representative branch-coverage excerpt from `TestComputeWeightUpdate()`, showing the principal reward, penalty, neutral, floor and ceiling update paths

Appendix D. Baseline 3 Reasoning Logs

```

FULL OUTPUT - PIPELINE vs BASELINE 3 (reasoning comparison)
=====
EURUSD 1h
PIPELINE stance=bullish   conf=0.648   conflict=low
Reasoning : All models indicate a bullish stance for the EURUSD pair on the 1h timeframe. The Custom GNN model and the WOLD chart pattern detection both predict a bullish trend, while retail positioning shows a slight bias towards long positions. However, news sentiment is slightly bearish, which may carry reduced influence g
BASELINE3 stance=bullish   conf=0.780   conflict=medium
Reasoning : The bullish bias in the news, supported by the European Central Bank's tightening hopes, as well as the Euro's stability against other currencies such as the Canadian Dollar. The slight uptrend in the most recent candle and the retail sentiment slightly favoring long positions also contribute to a bullish stan
ce.

GBPUSD 1h
PIPELINE stance=bullish   conf=0.620   conflict=medium
Reasoning : Despite the bearish signal from the GNN model and slightly bearish news sentiment, the Head and shoulders bottom chart pattern and contrarian retail positioning suggest a potential bullish trend. However, the conflict level is medium due to the imbalance between the models' predictions.
BASELINE3 stance=bullish   conf=0.780   conflict=medium
Reasoning : The bullish bias in the news, combined with the slight uptrend in the chart and the consolidation in the time-series, suggests potential for further upside. However, the heavy retail short positioning indicates a conflict that should be considered.

USDJPY 1h
PIPELINE stance=bullish   conf=0.728   conflict=low
Reasoning : All four models indicate a bullish stance for USDJPY. The chart pattern and time-series analysis suggest a potential reversal, while the news sentiment and retail positioning data provide some contrast but do not significantly impact the overall directional agreement.
BASELINE3 stance=bullish   conf=0.780   conflict=medium
Reasoning : The bullish bias from the headlines and the positive price action on the chart suggest a potential upward movement. However, the high level of retail shorts indicates a potential conflict that should be monitored closely.

AUDCAD 1d
PIPELINE stance=bullish   conf=0.679   conflict=high
Reasoning : Despite the bearish chart pattern and retail positioning, both the news sentiment and GNN model indicate a bullish outlook. However, the high conflict level suggests caution and potential for market reversal. The GNN signal carries reduced influence due to its lower confidence level.
BASELINE3 stance=BHBRN   conf=0.000   conflict=?
Reasoning : Expecting ' '. delimiter: line 10 column 4 (char 489)

USDCAD 1d
PIPELINE stance=bullish   conf=0.846   conflict=low
Reasoning : All four models indicate a bullish stance for USDCAD on the 1d timeframe. The Head and shoulders bottom pattern, GNN model prediction, and retail positioning data all support a potential reversal to a bullish trend. However, it's important to note that the GNN model confidence is below 1.0, carrying reduced influe
nce for this pair.
BASELINE3 stance=BHBRN   conf=0.000   conflict=?
Reasoning : Expecting ' '. delimiter: line 10 column 4 (char 593)

```

Figure D.1: Complete Baseline 3 reasoning logs generated by eval_orchestrator.py for the five evaluated currency pairs

Appendix E. Reasoning Coherence Scoring Rubric

```

=====
COHERENCE SCORING - MANUAL STEP (pipeline vs Baseline 3 only)
=====

Score each reasoning output 1-5:
5 - References specific signals and explains interactions clearly.
4 - Relevant and mostly grounded. Minor gaps or vagueness.
3 - Plausible but generic. Could apply to any market situation.
2 - Partially contradicts signals or ignores key modalities.
1 - Incoherent, factually wrong, or completely generic.

```

Figure E.1: Reasoning coherence scoring rubric used by eval_orchestrator.py to evaluate LLM outputs

Appendix F. Full Ten-Run Consistency Results

RESULTS TABLE				
Run	Stance	Confidence	Conflict	Time (s)
1	bullish	0.638	low	64.0
2	bullish	0.648	low	37.4
3	bullish	0.648	low	37.3
4	bullish	0.648	low	35.6
5	bullish	0.648	low	35.6
6	bullish	0.648	low	33.4
7	bullish	0.648	low	33.2
8	bullish	0.648	low	39.8
9	bullish	0.648	low	30.0
10	bullish	0.648	low	28.9
SUMMARY				
Successful runs : 10 / 10				
Stance distribution:				
bullish 10x (100%)				
Consistency rate : 100% (bullish was most common)				
Confidence stats:				
Mean : 0.647				
StdDev : 0.003				
Min : 0.638				
Max : 0.648				
Conflict distribution:				
low 10x (100%)				

Figure F.1: Full output from the 10-run EURUSD (1h) consistency evaluation from eval_orchestrator.py

Appendix G. User Acceptance Testing Materials

G.1. User Acceptance Questionnaire

1. The fusion output (stance, confidence, conflict level) was easy to understand at a glance. *

	1	2	3	4	5	
Disagree	☐	☐	☐	☐	☐	Agree

2. The reasoning text explained why the system reached its conclusion, not just what the conclusion was. *

	1	2	3	4	5	
Disagree	☐	☐	☐	☐	☐	Agree

3. Where modalities disagreed, the dashboard made it clear which signals were driving or working against the final stance. *

	1	2	3	4	5	
Disagree	☐	☐	☐	☐	☐	Agree

3. Open-ended: What, if anything, did you find confusing, difficult to use, or in need of improvement?

Your answer

Figure G.1: User acceptance testing questionnaire

G.2. Likert-Scale Results

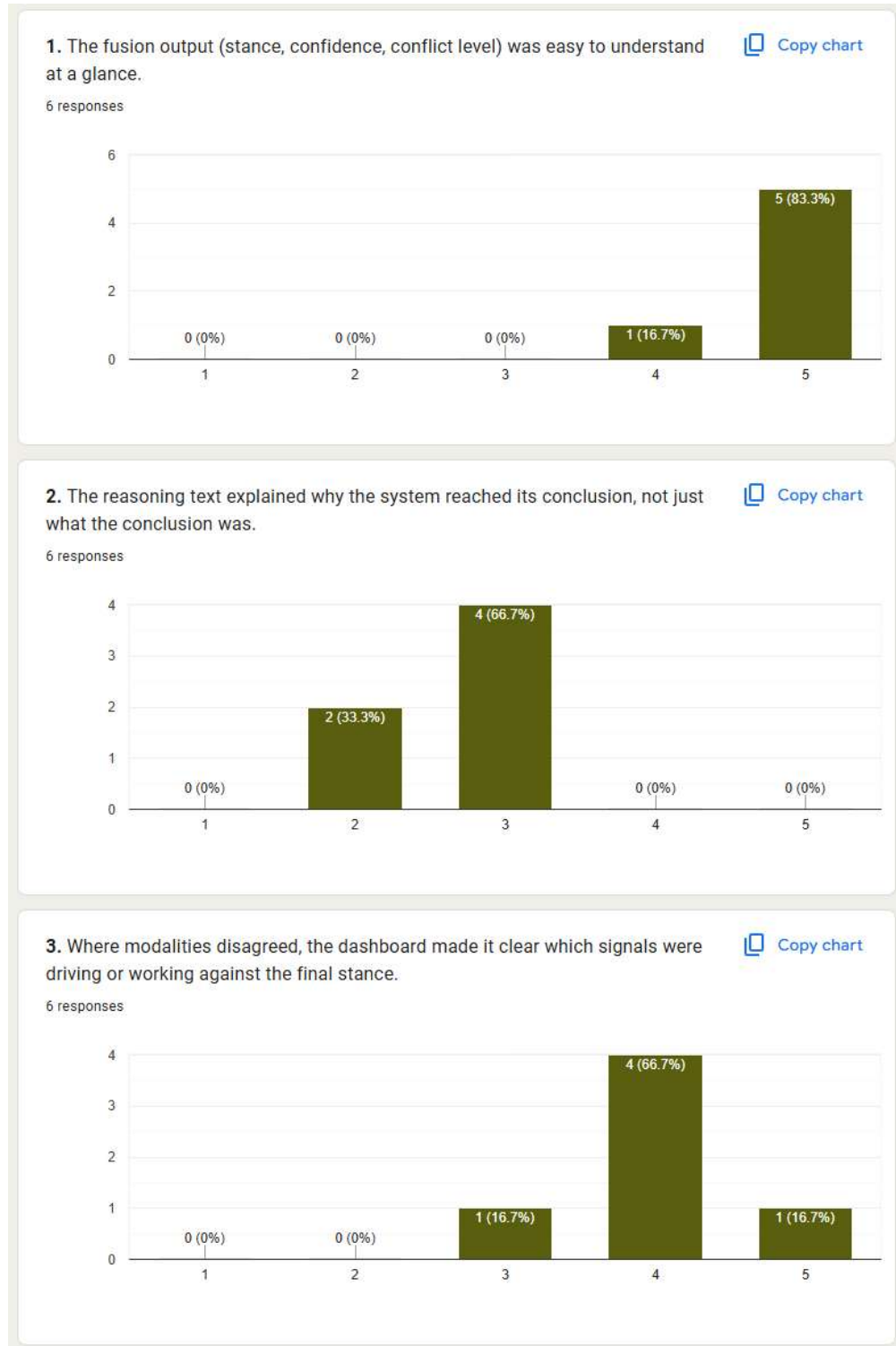


Figure G.2: Summary of participant responses to the three Likert-scale questions

G.3. Open-Ended Responses

3. Open-ended: What, if anything, did you find confusing, difficult to use, or in need of improvement?

6 responses

The confidence level is shown per analysis but I don't know what it means or what lead to that confidence level, transparency can be improved on this.

After trying for a couple times, i realise i cannot go back and add the feedback on whether it was right or wrong if i refresh the page or make another analysis. I dont think we will be able to immediately give feedback as we have to wait the trade out for the result

i think there is an issue that i cant update my old analysis feedback unless i do it immediately... It should allow us to change the result of analysis as we may accidentally close the window/browser

The reasoning output may be too short and does not give enough explanation

Reasoning seems brief, i do understand that it is there as a 1-2 lines to compile the analysis, but if it is so brief and shallow it provides no real insights. Making it feel slightly like deadspace. But still a nice piece for people who want a watered-down summary.

The recent analysis section only show a limited amount of 5 previous analysis, if i want to look at older analysis i cannot do that. Its weird that the weight chart allow us to view ALL analysis but i have no way of actually looking at each individual history on the left panel.

Figure G.3: Collated open-ended participant responses

8. References

- Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. arXiv preprint, arXiv:1908.10063. <https://arxiv.org/abs/1908.10063>
- Caporale, G.M. & Plastun, A. (2023). Seven Pitfalls of Technical Analysis. Brunel University London, Department of Economics and Finance. <https://www.brunel.ac.uk/economics-finance-and-accounting/research/pdf/2301-Jan-GMC-Technical-Analysis.pdf>
- Fazlollahi, B., Parikh, M.A. & Verma, S. (1997). Adaptive decision support systems. *Decision Support Systems*, 20(4), pp.297–315. <https://www.sciencedirect.com/science/article/abs/pii/S0167923697000146>
- Fjellström, C. (2022). Long Short-Term Memory Neural Network for Financial Time Series. arXiv preprint, arXiv:2201.08218. <https://arxiv.org/abs/2201.08218>
- Hangloo, S. & Arora, B. (2025). Multimodal fusion techniques: Review, data representation, information fusion, and application areas. *Neurocomputing*, 649, 130827. <https://www.sciencedirect.com/science/article/abs/pii/S0925231225014997>
- Holopainen, M. & Sarlin, P. (2016). Toward Robust Early-Warning Models: A Horse Race, Ensembles and Model Uncertainty. arXiv preprint, arXiv:1501.04682. <https://arxiv.org/abs/1501.04682>
- Huang, Y., Du, C., Xue, Z., Chen, X., Zhao, H. & Huang, L. (2021). What Makes Multi-modal Learning Better than Single (Provably). *Advances in Neural Information Processing Systems (NeurIPS)*, 34. arXiv:2106.04538. <https://arxiv.org/abs/2106.04538>
- Kim, A.G., Muhn, M. & Nikolaev, V.V. (2024). Financial Statement Analysis with Large Language Models. University of Chicago, Booth School of Business. https://www.bayes.citystgeorges.ac.uk/__data/assets/pdf_file/0009/799794/Alex-Kim_Financial_Statement_Analysis_with_Large_Language_Models__2024_-6.pdf
- Lee, S.I. & Yoo, S.J. (2019). Multimodal Deep Learning for Finance: Integrating and Forecasting International Stock Markets. arXiv preprint, arXiv:1903.06478. <https://arxiv.org/abs/1903.06478>
- Lin, W., Wang, M., Tsai, C. & Hsu, J. (2026). Image-Based Deep Learning Models for Stock Predictions: Combining Line, Candlestick, and Bar Charts. *Journal of Forecasting*. <https://onlinelibrary.wiley.com/doi/10.1002/for.70099>
- Mino, D. & Williamson, C. (2025). Sentiment and Volatility in Financial Markets: A Review of BERT and GARCH Applications during Geopolitical Crises. University College Cork. arXiv:2510.16503. <https://arxiv.org/html/2510.16503v1>
- Roetzel, P.G. & Fehrenbacher, D.D. (2019). On the Role of Information Overload in Information Systems Success: Empirical Evidence from Decision Support Systems. 40th International Conference on Information Systems, Munich, 15–18 December 2019. <https://research.monash.edu/en/publications/on-the-role-of-information-overload-in-information-systems-is-suc/>

Wu, S., Wang, J., Guan, Z., Zhao, L., Song, X., Ying, X., Yu, D., Wang, J., Zhang, H., Pak, M., He, Y., Xin, Y., Wang, J. & Shi, T. (2025). MountainLion: A Multi-Modal LLM-Based Agent System for Interpretable and Adaptive Financial Trading. arXiv preprint, arXiv:2507.20474.
<https://arxiv.org/abs/2507.20474>

Zhang, W., Zhao, L., Xia, H., Sun, S., Sun, J., Qin, M., Li, X., Zhao, Y., Zhao, Y., Cai, X., Zheng, L., Wang, X. & An, B. (2024). A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist. Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Barcelona, Spain. arXiv:2402.18485.
<https://arxiv.org/abs/2402.18485>